Project Handover Document

Project Name

A GraphRAG-Powered Personalized Learning Platform for Medical Students

Project Contributors

- Michael Eko Hartono Gunawan
- Ghazali Ahlam Jazali
- Darrell Cornelius Rivaldo

Stakeholders

- dr. Florence Pribadi, M.Sit.
- dr. Maria Jessica Rachman, M.Si.

Project Period

Start Date : 15-07-2026
End Date : 21-08-2026

Deliverable Checklist

Item	**Location**
Raw Data	`https://accessmedicine.mhmedical.com/book.aspx?bookid=3634#304624809`
Preprocessed Data	`https://github.com/medical-uc/knowledge-graph-pipeline/releases/tag/v0.9.4`
Model Experiment Notebooks	https://github.com/medical-uc/intelligent-tutoring-system-for-medical-student/tree/main/notebooks
Apps Source Code	`https://github.com/orgs/medical-uc/repositories`
License	`https://accessmedicine.mhmedical.com/ss/terms.aspx`
Access Ownership	`https://github.com/medical-uc/Handover-Document/blob/main/access_ownership.zip`

Executive Summary

Medical education presents severe cognitive challenges for first-year medical students who must master thousands of complex, interconnected concepts across dense foundational textbooks (*Harper's Illustrated Biochemistry*). Because traditional classroom instruction progresses at a uniform pace, students with diverse baseline proficiencies often develop unresolved knowledge gaps and persistent misconceptions. At the same time, authoring high-yield, clinically rigorous practice questions manually presents a major institutional bottleneck, while standard Large Language Models (LLMs) are prone to factual hallucinations, ambiguous distractors, and answer leakage when generating medical assessments without formal semantic constraints.

To resolve these challenges, we built **MediQuiz**, an Intelligent Tutoring System (ITS) and adaptive study platform designed with a decoupled architecture that strictly isolates offline AI/ML knowledge extraction from a zero-inference live serving tier. The delivered system comprises: (1) an offline GraphRAG and knowledge engineering pipeline that combines layout OCR (`MinerU`), deterministic cleaning (`post_process_book`), structural document rewriting (`document-rewriter`), biomedical entity linking to UMLS/SNOMED CT (scispaCy, SapBERT/FAISS), and 27-relation medical ontology extraction (HuatuoGPT-o1-72B) with open-world distractor safety and batch stem phrasing (`knowledge-graph-pipeline`) to synthesize verified, curriculum-grounded MCQs; (2) a high-throughput FastAPI backend backed by PostgreSQL for immutable question banks and Neo4j for event-sourced student interaction graphs; and (3) a native iPadOS application (SwiftUI) featuring adaptive practice quizzes, confidence-based immediate feedback, dual-track spaced-repetition flashcards, interactive curriculum mastery progress bars, and gamified study nudges.

For personalized learning, MediQuiz implements a confidence-weighted Bayesian Knowledge Tracing (BKT) engine and spaced-repetition scheduling that execute instantaneously on-device. By calibrating BKT parameters globally across 10,480 interaction sequences using the Baum-Welch Expectation-Maximization (EM) algorithm, next-step correctness prediction improved from an uncalibrated heuristic baseline of **0.5658 to 0.6819 ROC-AUC (+20.5% relative gain)** and reduced **Log-Loss from 0.7491 to 0.6404 (14.5% error reduction)**. Furthermore, multi-graph blocklisting, deterministic relation guards, and indefinite stem framing prevent distractor ambiguity and factual hallucination—providing medical students with an adaptive, clinically grounded, and scalable personal tutor.

Project Overview

Problem Statement

First-year medical students at Ciputra University differ in their strengths, weaknesses, and learning pace. Although a standardized curriculum ensures that every student receives the same foundational learning materials, lectures are typically delivered at a uniform pace and level of difficulty. This limits the ability of classroom instruction to accommodate individual learning differences, leaving some students with unresolved knowledge gaps. Therefore, additional personalized learning activities are needed to support each student's specific learning needs.

Solution

To address the varying strengths, weaknesses, and learning pace of first-year medical students, we propose an adaptive learning system that continuously estimates each student's mastery of medical concepts and recommends personalized practice questions and flashcard reviews. By tailoring learning content to individual needs, the system helps students focus on concepts they have not yet mastered, improving learning efficiency and supporting more effective knowledge acquisition.

Result

As the solution to personalized medical learning, we developed **MediQuiz**, an Intelligent Tutoring System (ITS) and adaptive study application built natively for iPadOS. The application serves as an interactive study companion for first-year medical students, translating dense medical textbook curricula into personalized practice quizzes, spaced-repetition flashcards, and real-time knowledge tracking.

Application Overview & Core Features

MediQuiz provides an intuitive and distraction-free learning environment designed to support self-directed study through the following features:

- **Adaptive Practice Quizzes:** Students can generate tailored practice quizzes from specific medical subjects (e.g., Physiology, Biochemistry, Histology, Anatomy) or construct custom multi-topic study sessions. Questions feature multiple difficulty levels ranging from fundamental recall to clinical case vignettes.
- **Confidence-Based Immediate Feedback:** While answering questions, students indicate their self-reported confidence level (*Confident*, *Unsure*, or *Guessing*). Upon selection, the application provides instant verification along with comprehensive clinical explanations to reinforce underlying concepts and correct misconceptions.
- **Spaced-Repetition Flashcard Decks:** Students can review core medical facts through interactive flashcards with flip animations. By rating their recall difficulty (*Again*, *Hard*, *Good*, *Easy*), the platform dynamically calculates optimal future review dates to maximize long-term retention.

- **Curriculum Exploration & Mastery Tracking:** An interactive subject directory provides students with visual progress bars indicating their mastery level across each topic and sub-topic, making it effortless to identify weak areas and track learning progression over time.
- **Smart Dashboard & Study Nudges:** The main dashboard delivers actionable study recommendations ("nudges") that prioritize urgent review items and topics requiring reinforcement. Daily streak counters and gamified energy rewards encourage consistent daily practice.
- **Review History & Bookmarking:** Students can bookmark challenging questions for focused revision and review past quiz attempts and historical performance summaries.

Model Outcomes & Validation Results

The initial evaluation and operational benchmarks of the AI pipeline and adaptive models showed the following results:

- **Optimized Knowledge Tracing (BKT):** Following parameter optimization using the Baum-Welch Expectation-Maximization (EM) method, the Bayesian Knowledge Tracing model achieved an improved **ROC-AUC of 0.6819** and a **log-loss of 0.6404** for next-step correctness prediction (improving from the 0.5658 ROC-AUC and 0.7491 log-loss heuristic baseline), providing a reliable signal for estimating topic mastery and surfacing student knowledge gaps.
- **Graph-Grounded Question Synthesis:** Leveraging the biomedical knowledge graph pipeline (`knowledge-graph-pipeline`), questions are synthesized directly from asserted semantic triples across a 27-relation medical ontology. Each item anchors to a verified source sentence as an auditable rationale, using indefinite stem framing to prevent open-world factual inaccuracies.
- **Multi-Tier Distractor Safety & Semantic Guardrails:** To ensure distractors are plausible yet unambiguously incorrect, the pipeline enforces a multi-tier defense: filtering against global relation blocklists (asserted, inferred, and flagged edges), excluding cross-related concepts, and applying deterministic relation guards that prevent clinically inverted claims (such as distinguishing nutrient deficiencies from the nutrients themselves).

AI/ML Solution Technical Overview

Data Collection & Preprocessing

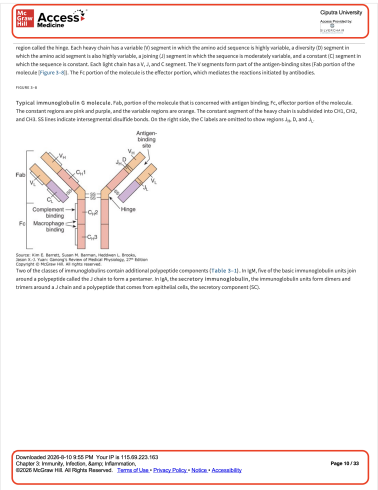
Data Sources

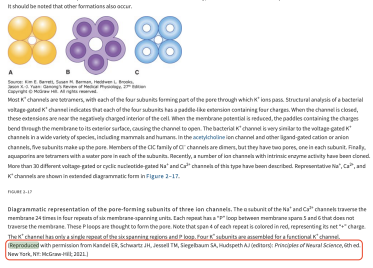
- **Medical Textbooks (`data/textbook/`):**
  - **Source & Format:** AccessMedicine by McGraw-Hill (PDF, 44 chapters)
  - **Titles:** *Harper's Illustrated Biochemistry*
  - **Content Structure:**
    - Chapter titles, headings, and sub-headings
    - Paragraph body text and nested list structures
    - Tables with captions and footnotes
    - Diagrams and flowcharts
    - Inline mathematical equations

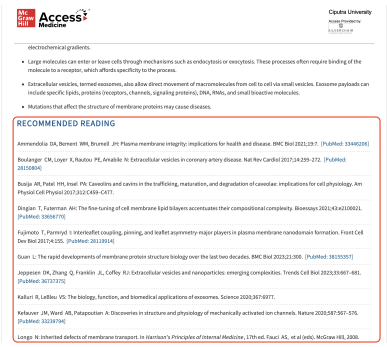
Data Cleaning

Raw PDFs are first extracted by **MinerU** (in an isolated Python environment via `notebooks/parse_books.ipynb`), which produces a structured `*_middle.json` block file and associated `*_content_list.json` per chapter. The `src/post_process_book` module (in `intelligent-tutoring-system-for-medical-student`) then cleans this MinerU output to produce the final text corpus.

- Removing irrelevant content blocks: image blocks, running headers/footers, page numbers, source attribution lines, "reproduced/adapted with permission" phrases, publisher citation parentheticals, and entire *Recommended Reading* / *References* sections.







- Preserving all paragraph text, list structures, table bodies, inline figure references (e.g. *Figure 1-1*), and mathematical equations in their original form.
- Outputs per chapter: `<basename>.cleaned.md` (full cleaned markdown) and `<basename>.page_map.json` (character-level and line-level page offsets used for downstream retrieval grounding).

Data Preprocessing

After cleaning, the `.cleaned.md` files pass through two sequential steps before knowledge-graph construction.

- **Document rewriting:** `rewrite_medical_md.py` (`document-rewriter` repository) reads each `.cleaned.md` chapter and re-emits it as a structurally tagged markdown file. Body paragraphs are carried over verbatim into `<con>` elements; PDF-extraction artefacts (non-breaking spaces, soft hyphens, mid-sentence page-break splits) are repaired. Inference is limited to one **HuatuoGPT-o1-72B** call per chapter for the `<topic>` element, which has no source counterpart.
- **Stage 1 parse:** The tagged markdown is consumed by Stage 1 of the `knowledge-graph-pipeline` repository (`medkg/stage1_parse.py`), which produces a tag-free normalised text file and a structured IR JSON carrying offset-anchored chunks, figures and passages. Chunk kinds are `prose`, `definition`, `clinical`, `summary` and `caption`; only the first four reach Stage 2 extraction.

Feature Engineering

- **Context enrichment:** The most specific section heading from each chunk's `section_path` is prepended to its text before any LLM call, preserving the governing medical concept even for short fragment chunks.
- **Medical triple extraction:** Stage 2 of the `medkg` pipeline makes one **HuatuoGPT-o1-72B** LLM call per chunk to extract structured `{subject, relation, object}` triples. Relations are constrained to a closed **27-relation taxonomy** defined in `medkg/ontology_medschool.json`: `treated_with`, `prevented_by`, `caused_by`, `finding_site`, `associated_with`, `present_in_patient`, `part_of`, `located_in`, `composed_of`, `connects_to`, `secretes`, `synthesizes`, `stores`, `transports`, `converts_to`, `catalyzes`, `regulates`, `stimulates`, `inhibits`, `requires`, `has_function`, `risk_factor_for`, `complication_of`, `diagnosed_by`, `adverse_effect_of`, `contraindicated_in`, `has_mechanism`. 14 deterministic relation guards then audit every extracted relation before assertion.
- **Distractor engineering:** A three-tier deterministic strategy selects plausible wrong answer options directly from the domain knowledge graph:
  - *Tier 1 (Same-Role Siblings):* Other tails of the same predicate occurring elsewhere in the corpus that share the answer's semantic label, preserving syntactic and semantic register.
  - *Tier 2 (Guard Confusions):* Concept pairs reflecting common clinical misconceptions, such as antonym pairs (e.g., hyper- vs. hypo-) or deviation node variants of the same concept (e.g., *iodine* vs. *iodine deficiency*).
  - *Tier 3 (Semantic Type Peers):* Concepts sharing the same ontology span label.

  Open-world distractor safety is enforced via a multi-graph blocklist: any concept connected to the head under the target predicate across asserted, inferred, or flagged/uncertain graphs is strictly excluded.
- **Domain Knowledge Graph:** `medkg/` implements an 8-stage pipeline that builds an RDF knowledge graph from the textbook markdown — scispaCy NER, SapBERT/FAISS

entity linking to UMLS/SNOMED CT, deterministic post-coordination modifier minting (`INSTANCE` nodes), and RDFS/OWL-RL reasoning. This enriches the semantic structure of extracted medical entities beyond what the triple-extraction LLM alone can capture.

Exploratory Data Analysis

- **Knowledge graph extraction yield:** The domain knowledge graph pipeline extracts and structures verified biomedical relations across medical textbook chapters, anchoring every triple to an auditable source sentence rationale.
- **Deterministic guardrail & distractor filtering:** The 14 relation consistency guards and multi-graph distractor blocklists audit all extracted candidates, demoting suspect relations to `polarity = "uncertain"` and eliminating distractor leakage or clinically inverted claims prior to question synthesis.
- **MinerU quality diagnostics:**

  `src/evaluation/mineru_quality.py` provides reference-free quality metrics for raw MinerU output, including garbage-character ratio, empty-block ratio, repeated-character runs, and per-block confidence scores. These were used to spot-check extraction quality across the five textbook PDFs.
- **Knowledge graph structure:** Student interaction data is stored in Neo4j as `(Student)-[:ANSWERED]->(Question)` edges, with a topic hierarchy modelled as `(Topic)-[:SUB_TOPIC_OF]->(Topic)` to support spaced-repetition scheduling and Bayesian Knowledge Tracing queries.

Model Experiment

All models are deployed in an **inference-only** setup — no fine-tuning or backpropagation during live serving. The list below documents each model's role, architectural hyperparameters, operational runtime configuration, fallback defaults, and validation mechanisms across the pipeline:

- **HuatuoGPT-o1-72B (4-bit)** (*Medical Reasoning LLM*)
  - **Role:** Document rewriting and semantic tagging (`rewrite_medical_md.py`); Stage 2 relation and modifier extraction (`medkg/stage2_extract.py`); Stage 3 ambiguous concept reranking (`medkg/stage3_link.py`).
  - **Configuration & Hyperparameters:**
    - *Checkpoint & Backend:* `mlx-community/HuatuoGPT-o1-72B-4bit` served via an OpenAI-compatible vLLM/MLX endpoint (`DEFAULT_BASE_URL = "http://jupyter-02.aml1.id.iosda.org:8888"`, default fallback `http://localhost:8888`).
    - *Rewriter Sampling:* Section-specific temperatures: $T_{\text{topic}} = 0.30$, $T_{\text{caption}} = 0.15$ (strict factual alignment), $T_{\text{objectives}} = 0.35$, $T_{\text{summary}} = 0.35$, fallback $T = 0.40$; top-$p = 0.90$.
    - *Rewriter Token Limits:* max_tokens = 1024 for `<topic>` and `<cap>`; max_tokens = 2048 for `<obj>`, `<sum>`, and general completions.
    - *Stage 2 Extraction:* Greedy decoding ($T = 0.0$, max_tokens = 4096) with vLLM guided JSON decoding (`extra_body={"guided_json": schema}`).

    * *Stage 3 Disambiguation:* Greedy decoding ($T = 0.0$, max_tokens = 24) with guided choice (`extra_body={"guided_choice": choices}`), cached per `(mention, choices)` tuple.
    * *Execution & Resilience:* 900 s timeout (rewriter) / 300 s timeout (Stage 2); 4 retries with exponential backoff; 4 concurrent worker threads; SHA-256 payload disk caching; client-side chain-of-thought stripping (`split_reasoning` discarding `## Thinking` and `<think>` traces); circuit breaker halting after 5 consecutive failures (`BackendUnavailable`); `-offline` rule-based fallback.
  - **Validation:** Tag-completeness checks; exact-match chunk text grounding; Pydantic schema validation (`Triple`, `Modifier`); closed 27-relation taxonomy guards.

- **SapBERT** (*Biomedical Sentence Transformer*)
  - **Role:** Stage 3 entity linking to UMLS/SNOMED CT concepts (`medkg/stage3_link.py`).
  - **Configuration & Hyperparameters:**
    * *Checkpoint & Pooling:* `cambridgeltl/SapBERT-from-PubMedBERT-fulltext` ($d = 768$); $[CLS]$ token representation from `last_hidden_state`; unit L_2-normalization.
    * *Index & Search:* FAISS `IndexFlatIP` (Exact Inner Product on normalized vectors $\equiv$ Cosine Similarity); candidate retrieval depth $k = 20$; sequence length capped at 32 tokens; batch size 256.
    * *Thresholds:* Cosine similarity floor ≥ 0.70 (`LINK_CONFIDENCE_FLOOR = 0.70`); LLM reranker invoked only when top-2 candidate margin $(\text{score}_1 - \text{score}_2) < 0.05$ (`RERANK_MARGIN = 0.05`).
    * *Concurrency Safety:* Native thread pinning (`MEDKG_NATIVE_THREADS = 1`) across OpenMP, PyTorch, and FAISS runtimes to prevent macOS Darwin/BLIS segmentation faults.
    * *Semantic Typing:* UMLS `MRSTY.RRF` crosswalk (`semantic_types.json`) applying `SEMANTIC_TYPE_PRIORITY` to refine coarse NER labels into canonical ontology span labels.
  - **Validation:** Link-confidence distribution; CUI collision detection; semantic type compatibility checks.

- **Qwen3.5-27B / Qwen3.5-9B-OptiQ** (*LLM*)
  - **Role:** MCQ question-stem phrasing, stylistic rephrasing, and validation guardrails (`knowledge-graph-pipeline/medkg/mcq.py`).
  - **Configuration & Hyperparameters:**
    * *Checkpoint & Backend:* `Qwen/Qwen3.5-27B` or `mlx-community/Qwen3.5-9B-OptiQ-4bit` via OpenAI-compatible endpoint (`http://localhost:8888/v1`); $T = 0.0$, max_tokens = 4096, guided JSON schema decoding.
    * *Batching & Limits:* Batch size of 10 items per phrasing pass (`phrase_items`); maximum stem length capped at 300 characters; 5 options per question (1 key + 4 distractors).
    * *Distractor Safety Guardrails:* 3-tier open-world exclusion (asserted/inferred/flagged triple blocklist; alternate head-edge prohibition; indefinite stem framing); negative-stem regex guard (`_NEGATIVE_RE`) rejecting negative phrasing to prevent open-world fallacies.

    * *Stem Rewriting Safeguards:* Checks that rephrased stems do not leak the key or distractors and end with a question mark. Rejections fall back deterministically to rule-based template stems (`accept_stem`).
  - **Validation:** Negative-stem guard; stem character ceiling; no-option-leakage validation; deterministic template fallback.

- **Qwen2.5-VL-7B-Instruct & 3B-Instruct** (*Vision-Language Model*)
  - **Role:** Textbook diagram/figure visual captioning (`src/captioning/qwen_vl.py`), full-page OCR transcription (Thread B), and exam PDF question extraction.
  - **Configuration & Hyperparameters:**
    * *Precision & Placement:* `torch.bfloat16`; two-stage load (CPU → MPS/CUDA) bypassing Apple Silicon MPS fp16 kernel crashes; lazy instantiation with explicit garbage collection and cache clearance (`torch.mps.empty_cache()`).
    * *Visual Captioning:* max_new_tokens = 512, deterministic greedy decoding (do_sample = False), grounded with MinerU OCR labels.
    * *Page Transcription:* max_new_tokens = 2048, max_pixels = 1,280,000 (1.28M px) bounding vision prefill token overhead on MPS.
    * *Image Relevance:* max_new_tokens = 5, max_pixels = 480,000, single-token classification (`SEMANTIC` vs. `DECORATIVE`).
  - **Validation:** Visual grounding against MinerU OCR text labels; single-word relevance classification output validation.

- **scispaCy & NegEx** (*Biomedical NER & Negation Detection*)
  - **Role:** Stage 2 entity recognition and negation detection (`medkg/stage2_extract.py`).
  - **Configuration:** Multi-model pipeline (`en_ner_bc5cdr_md` for diseases/chemicals, `en_ner_bionlp13cg_md` for anatomy/cells/proteins); `AbbreviationDetector` and `Negex` over all entity types; longest-span-first entity merging.
  - **Validation:** Cross-model span deduplication; ontology label mapping per `NER_LABEL_MAPS`.

- **all-MiniLM-L6-v2** (*Sentence Transformer*)
  - **Role:** Semantic boundary chunking of unified text streams (`src/ingestion/semantic_chunk.py`).
  - **Configuration:** Cosine distance threshold calculated at the 95.0 percentile over each document; hard fallback split at 6000 characters.
  - **Validation:** Distributional percentile adaptation per textbook writing style.

- **Bayesian Knowledge Tracing & Spaced Repetition Algorithms** (*Probabilistic & Cognitive Models*)
  - **Role:** Real-time student concept mastery tracking and personalized quiz/flashcard review scheduling (`src/quiz/`, `src/flashcards/`, `BKTStore.swift`, `notebooks/knowledge_tracing.ipynb`).
  - **Model Architecture & Confidence Conditioning:**
    * *2-State Knowledge Engine:* Tracks latent mastery (learned vs. unlearned) per (student, leaf topic) pair with a unidirectional transition structure assuming mastery acquisition without immediate forgetting.

    * *Confidence-Weighted Evidence:* Integrates self-reported confidence (*confident*, *unsure*, *guessing*) directly into the emission probabilities. This prevents lucky guesses from falsely indicating mastery while treating confident errors as critical misconceptions that substantially depress the estimated mastery score.
    * *Hybrid Client-Server Deployment:* The population-wide BKT parameters are estimated server-side and cached by the iOS app via `GET /quiz/mastery/params`. This enables `BKTStore.swift` to execute instantaneous, lightweight $O(1)$ mastery updates on-device during offline or online quiz attempts, syncing results back to Neo4j via `PUT /quiz/mastery`.
- **Optimization via Baum-Welch Expectation-Maximization (EM):**
    * *Overcoming Heuristic Limitations:* Hand-picked prior, transition, slip, and guess values acted as an artificial performance ceiling, yielding near-chance predictive capability (ROC-AUC = 0.5658, log-loss = 0.7491).
    * *Global Sequence Pooling:* Because individual topic histories are sparse (median 6 attempts per student-topic pair), fitting parameters per-topic risks severe overfitting and degenerate parameter states. We formulated Baum-Welch EM to pool all 1,554 chronological interaction sequences across 43 students and 56 topics (10,480 total quiz attempts) into a unified, robust parameter estimation pass.
    * *Iterative Learning Dynamics:* The Expectation step runs forward-backward passes across complete chronological interaction sequences to evaluate latent mastery state trajectories over time. The Maximization step aggregates these expected state transitions and observation counts to re-estimate initial mastery priors, transition rates, and confidence-stratified slip and guess probabilities until log-likelihood convergence (converged in 14 iterations).
    * *Learned Parameter Adjustments:* The data-driven optimization adjusted the initial mastery prior from 0.30 to 0.4033, smoothed transition rates from 0.10 to 0.0551, and aligned emission rates with empirical student response behaviors across all confidence tiers.
- **Predictive Evaluation & Performance Gains (Next-Step P(Correct)):**
    * *Heuristic Baseline (Before EM):* ROC-AUC = **0.5658**, Log-Loss = **0.7491** (barely above chance).
    * *Baum-Welch EM Optimized (After EM):* ROC-AUC = **0.6819** (+**0.1161** / +**20.5% relative gain**), Log-Loss = **0.6404** (**-0.1087** / **14.5% error reduction**).
    * *Personalization Impact:* This marked improvement transforms BKT from an unreliable heuristic into an effective ranking signal for identifying genuine knowledge gaps.
- **Personalization Scheduling & Pedagogical Diagnostics:**
    * *Quiz Spaced Repetition:* Confidence-weighted interval progression on `(:Student)-[:REVIEWING]->(:Question)`: confident correct answers double review intervals ($1 \to 2 \to 4 \to \cdots \leq 60$ days), whereas incorrect or unsure attempts reset streaks to 1 day.
    * *Flashcard Spaced Repetition:* 4-point rating schedule on `(:Student)-[:HAS_FLASHCARD]->(:Flashcard)`: *Again* (1 day), *Hard* (1.2×), *Good* (2.0×), and *Easy* (2.5×), capped at 60 days.
    * *Pedagogical Gap Flagging:* Weak-topic queries distinguish unattempted topics (`low_evidence`, < 3 attempts, preventing premature weak-topic assertions) from persistent misconceptions (`stuck`, ≥ 3 attempts with average question retry counter ≥ 2.0 while mastery remains low, prompting alternative explanatory materials rather than simple repetition).

Conclusion

- **Final system architecture:** MinerU PDF extraction → `post_process_book` cleaning (`.cleaned.md` + `.page_map.json`) → `document-rewriter` (HuatuoGPT-o1-72B tagging) → `medkg` Stages 1–8 (parse → NER → entity linking → relation extraction → post-coordination → RDF assertion → OWL-RL reasoning → N-Quads) → Qwen3.5-27B MCQ generation pipeline. A decoupled FastAPI serving layer reads from PostgreSQL (`mcq_questions` table) and logs student interaction graphs in Neo4j, with **zero GPU/PyTorch runtime dependencies** in the live API.
- **Why it performs best:**
  - *Decoupled architecture:* The live API is fully CPU-bound; all heavy inference runs offline during ingestion and generation.
  - *High quality control:* Every served question is grounded in asserted biomedical knowledge graph triples and protected by multi-graph distractor blocklisting and deterministic relation guards before being committed to PostgreSQL.
  - *Graph-based student tracking:* Neo4j enables real-time spaced repetition and Bayesian Knowledge Tracing (BKT) based on per-topic mastery edges.
- **Comparison with alternative approaches:**
  - *Single-pass unconstrained LLM question generation:* Replaced by the graph-grounded triple extraction → open-world distractor safety → template/LLM phrasing pipeline to eliminate answer leakage, prevent open-world fallacies, and guarantee factual traceability.
- **Final performance metrics:**
  - *BKT knowledge tracing validation:* Next-step correctness prediction evaluated on 10,480 student interaction trajectories demonstrated significant gains from Baum-Welch EM parameter optimization: **ROC-AUC increased from 0.5658 (heuristic baseline) to 0.6819 (+20.5% relative gain)**, while **log-loss decreased from 0.7491 to 0.6404 (14.5% error reduction)**, confirming the effectiveness of EM fitting in eliminating heuristic parameter bottlenecks and tracking evolving mastery.
  - *Ingestion throughput:* `post_process_book` cleaner executes deterministically in sub-second time per PDF chapter.
  - *Question safety and traceability:* 100% of synthesized MCQs are anchored to verified textbook source sentences with multi-graph distractor verification.

AI/ML System Technical Overview

System Architecture

Architectural Philosophy: Decoupled Offline Pipeline and Live Serving

The system is strictly partitioned into two decoupled subsystems that interact exclusively through persisted storage contracts:

- **The Offline Content & Knowledge Pipeline (Heavy AI/ML):**
  - Ingests raw textbook PDFs via MinerU and cleans extraction outputs via `post_process_book`.
  - Formats faithful structurally tagged markdown via `document-rewriter` (HuatuoGPT-o1-72B).
  - Executes multimodal figure captioning (`Qwen2.5-VL-7B`), diagram leader-line detection (`MedGemma + UNet`), biomedical entity linking to UMLS/SNOMED CT (`scispaCy`, `SapBERT/FAISS`), and 27-relation knowledge graph construction (`knowledge-graph-pipeline`).
  - Synthesizes validated, graph-grounded MCQs with open-world distractor safety (`medkg/mcq.py`).
  - Requires CUDA/MPS GPU hardware, PyTorch, Transformers, FAISS, and isolated Python runtimes.
  - Produces clean, validated relational tables in PostgreSQL (`mcq_questions`) and semantic RDF graphs (`graph.nq`).
- **The Live Serving Layer (FastAPI + iPadOS):**
  - High-throughput, CPU-bound REST API built on FastAPI and Uvicorn.
  - **Zero PyTorch or Transformers runtime dependencies** — the serving layer never loads model weights, eliminating GPU hosting costs, cold-start latency, and out-of-memory crashes on web workers.
  - Backed by PostgreSQL for immutable question content and Neo4j for event-sourced student interaction history, spaced repetition scheduling, and topic mastery.
  - Real-time personalization executes directly on the student's iPad using an on-device Bayesian Knowledge Tracing (BKT) engine.

High-Level System Diagram

Service Layer & Component Decomposition

Backend Architecture (`intelligent-tutoring-system-for-medical-student`):

- `app.main`: Application entry point; installs middleware stack, CORS, SlowAPI rate limiting, and exception handlers.
- `app.auth_middleware`: Inspects every request against a strict regex path allowlist; resolves opaque Bearer tokens to `student_id` via Neo4j.

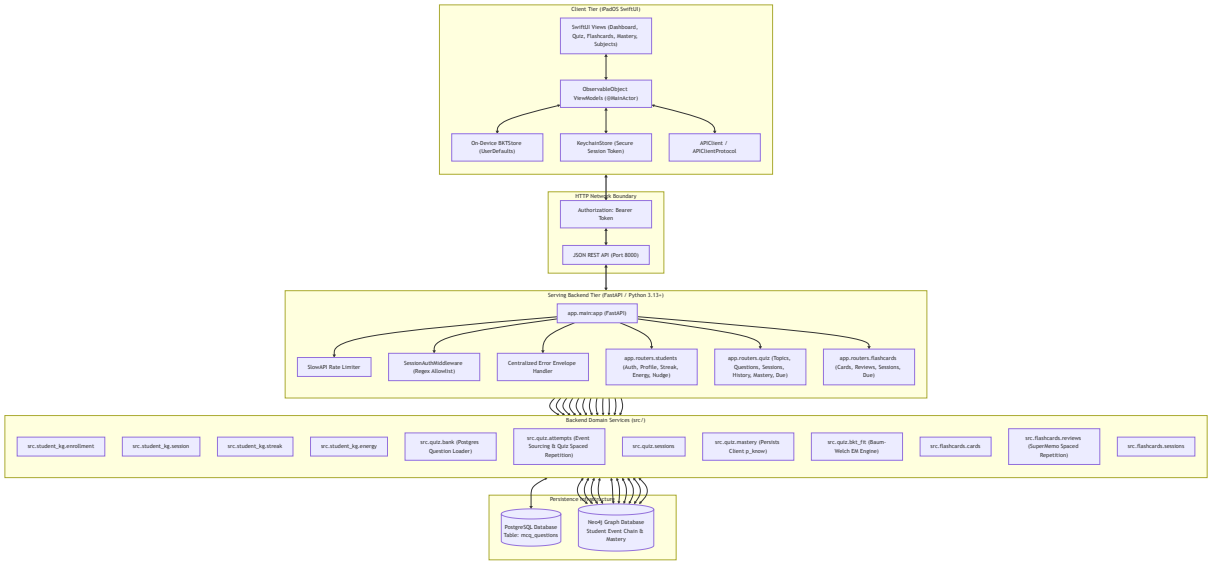


Figure 1: Full component decomposition: Client Tier, HTTP Network Boundary, Backend Tier, Domain Service Tier, and Persistence Infrastructure.

- `app.errors`: Formats all HTTP, validation, and driver exceptions into a unified contract: `{"error": {"code": "...", "message": "...", "details": [...]}}`.
- `src.student_kg`: Graph driver and node primitives for student records, streak calculations, gamified energy balances, and auth sessions.
- `src.quiz`: Database bank queries, session tracking, answer recording, spaced-repetition schedules, and Baum-Welch EM parameter fitting.
- `src.flashcards`: Flashcard generation from questions, 4-tier rating logging, and due-review queues.
- `src.captioning`: Vision-language captioning using Qwen2.5-VL-7B.
- `src.diagram_processing`: MedGemma 4B diagram classification and U-Net leader-line detection.
- `src.post_process_book`: Deterministic MinerU extraction cleanup, publisher attribution removal, dash normalization, and page mapping.

Frontend Architecture (`personalized-learning-ios`):

- **Pattern:** MVVM (Model-View-ViewModel) + Protocol-driven Networking.
- `App` **Layer:** Application lifecycle, root state routing (`ContentView`), and navigation container (`RootView`).
- `Core` **Layer:**
  - *Networking:* `APIClient` implementing `APIClientProtocol`, `APIConfig`, `APIError`, and custom ISO8601 date decoding.
  - *Persistence:* `KeychainStore` (secure credential store), `SessionManager` (session token façade), and `BKTStore` (on-device Bayesian Knowledge Tracing engine).
  - *UI:* Global typography, palette tokens (`Theme`), `SidebarView`, and `TopBarView`.

- `Features` **Layer:** Encapsulated feature domains — *Auth*, *Dashboard*, *Quiz*, *Flashcard*, *Subjects*, *History*, *Bookmark*, *Settings* — each containing dedicated Views, ViewModels, API extensions, and Models.

Storage & Graph Topology

The persistence model leverages PostgreSQL for relational data and Neo4j for event-sourced graph data.

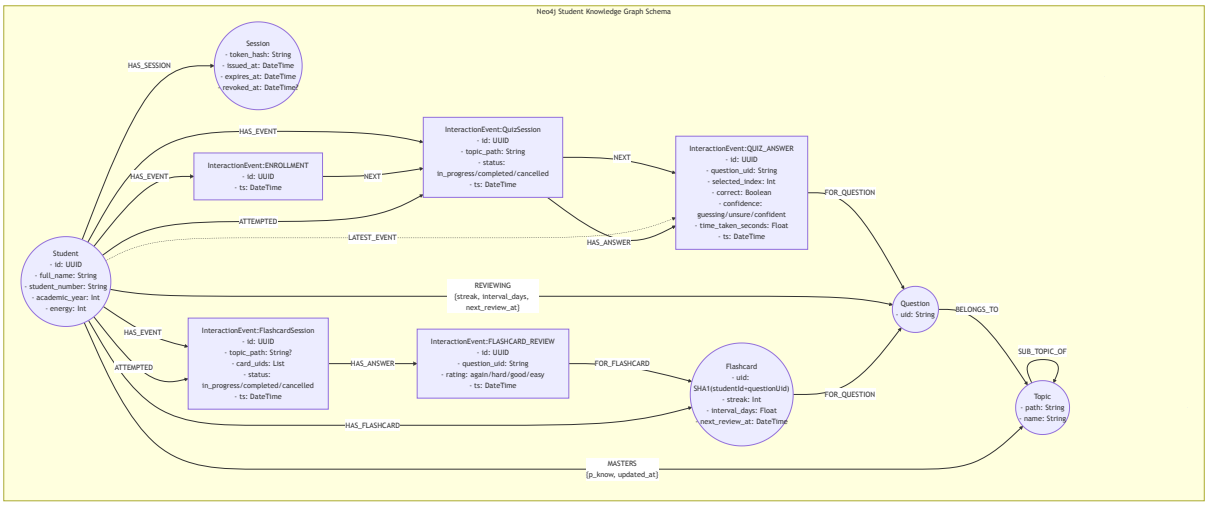


Figure 2: Neo4j Student Knowledge Graph Schema: nodes, relationships, and property contracts.

Key graph design rules:

1. **Event Sourcing:** Every student interaction creates an immutable `InteractionEvent` node linked in chronological order via `:NEXT` edges and grouped under `:QuizSession` / `:FlashcardSession` via `:HAS_ANSWER`.
2. **Mutable Schedules as Edges:** Spaced repetition schedules are stored directly on the `(Student)-[:REVIEWING]->(Question)` edge (quiz) and the `Flashcard` node (flashcards), updating deterministically upon each submission.
3. **Mastery Persistence:** The `:MASTERS` edge represents the student's on-device BKT estimate (p_{know}), synchronised asynchronously via `PUT /quiz/mastery`.
4. **Relational Question Isolation:** The `Question` node in Neo4j contains **only** `uid`. Stem text, options, explanations, and diagrams reside in PostgreSQL to protect graph traversal performance.

Security, Authentication & Session Model

Why Opaque Bearer Tokens (Not JWT)?

JWTs cannot be instantly revoked without a distributed blacklist. Storing a SHA-256 hashed token in Neo4j allows instantaneous server-side revocation on logout (`revoked_at = datetime()`) while keeping raw tokens out of the database.

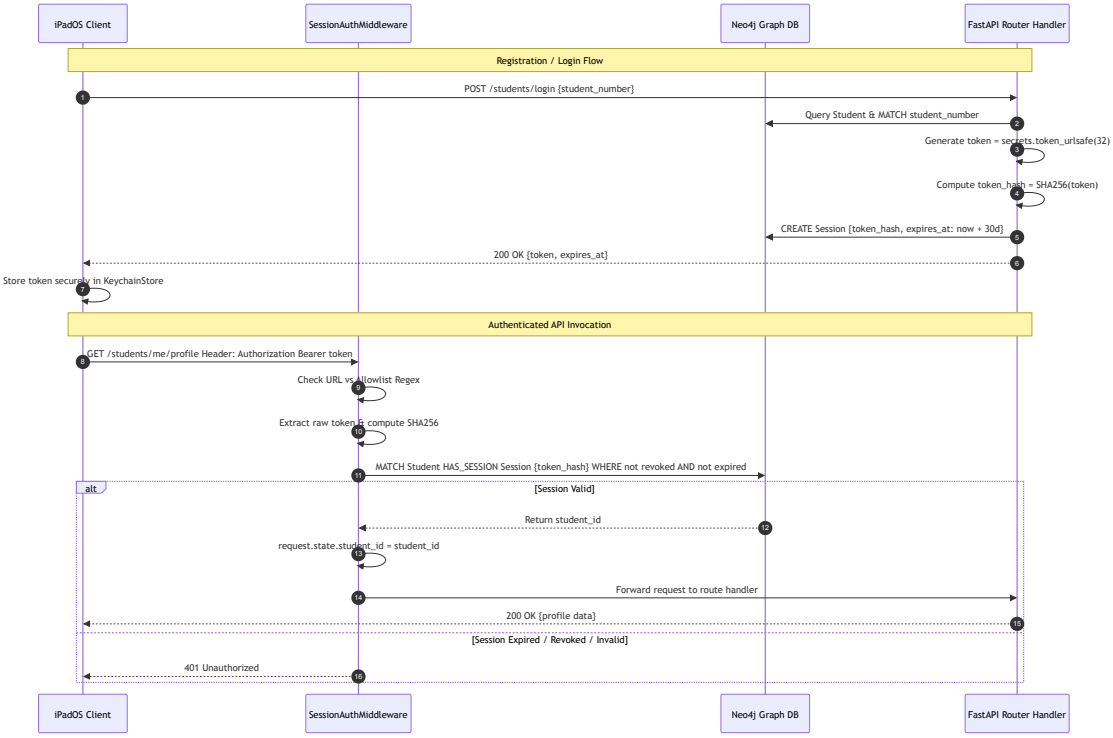


Figure 3: Opaque Bearer Token authentication sequence: registration/login flow and authenticated API invocation with `SessionAuthMiddleware`.

End-to-End Application Flows

Flow 1: Complete System End-to-End Dataflow

The application architecture coordinates three decoupled subsystems spanning offline content preparation, batch parameter estimation, and real-time interactive learning:

1. **Offline Content Ingestion & AI Generation:** Medical textbook PDFs are parsed via MinerU (`notebooks/parse_books.ipynb`) to extract layout blocks and content streams, cleaned deterministically via `src/post_process_book` to produce clean markdown and page mappings, and structured via `document-rewriter` into tagged study documents. The `knowledge-graph-pipeline` (`medkg`) executes an 8-stage biomedical knowledge graph extraction (scispaCy, SapBERT, HuatuoGPT-o1-72B, 14 relation guards, and OWL-RL reasoning). Graph-grounded MCQ synthesis (`medkg/mcq.py`) generates single-best-answer questions directly from asserted semantic triples with open-world distractor safety and batch LLM stem phrasing, populating PostgreSQL (`mcq_questions`). Multimodal textbook diagrams and figures are processed via `Qwen2.5-VL-7B` captioning and `MedGemma + UNet` leader-line detection.

2. **Bayesian Knowledge Tracing (BKT) Parameter Training:** Student interaction events (`:QUIZ_ANSWER`) recorded in Neo4j are pooled for global parameter fitting. The server executes the Baum-Welch Expectation-Maximisation (EM) algorithm (`src/quiz/bkt_fit.py`) to update population-wide transition, slip, and guess probabilities, exposing them through `GET /quiz/mastery/params`.

3. **Live Student Learning Interaction:** The iPadOS client authenticates with the FastAPI backend, caching session tokens in Keychain. Students engage with interactive Quiz and

Flashcard modules with sub-15 ms instant feedback, on-device Bayesian mastery updates, and asynchronous graph persistence to Neo4j.

Flow 2: Student Authentication & Session Lifecycle

The iPadOS client manages student identity through an opaque Bearer token architecture:

1. **Registration & Login:** The student submits credentials via `POST /students/login` or `POST /students/register`. The backend matches or creates the `:Student` node in Neo4j, mints a 32-byte cryptographic token (`secrets.token_urlsafe(32)`), computes its SHA-256 hash, and stores a `:Session` node with a 30-day expiration timestamp (`expires_at`).
2. **Secure Client Storage:** The raw token is returned to iPadOS and stored in `KeychainStore` using the Apple Security framework, while `SessionManager` publishes the active authentication state.
3. **Middleware Verification:** Subsequent HTTP requests provide the token via the `Authorization: Bearer <token>` header. `SessionAuthMiddleware` intercepts requests, computes the SHA-256 hash, and performs a single Cypher lookup against Neo4j to confirm the session is unrevoked and unexpired, injecting `student_id` into `request.state`.

Flow 3: Quiz Execution, Grading & Mastery Sync

The quiz lifecycle operates across four sequential phases designed to deliver instant feedback while guaranteeing asynchronous persistence (illustrated in Figure 5):

1. **Topic Configuration & Adaptive Weighting (Figure 5a, 5b):** The student selects up to three medical topics via `GET /quiz/topics`. When creating the session (`POST /quiz/topics/{path}/sessions`), the quiz builder generates a test set (e.g., 10 questions) with topic distribution weighted inversely proportional to the student's current Bayesian mastery estimate ($\propto 1 - p_{\text{know}}$), ensuring focused remediation on weak areas. Students can toggle *Review Mode* to receive immediate explanatory feedback after each item.
2. **Option Selection & Instant Grading (Figure 5c):** The student selects an answer option and self-reports a metacognitive confidence level (*Confident / Unsure / Guessing*). `POST /quiz/questions/{uid}/check` is a public, stateless endpoint that evaluates the option against an in-memory sanitized question bank—returning `correct`, `correct_index`, and a verified clinical `explanation` with textbook section provenance in sub-15 ms. Correct and incorrect choices are immediately color-coded in the UI (green checkmark vs. red cross).
3. **On-Device BKT Update:** Immediately upon evaluation, `BKTStore.record()` on iPadOS computes the updated Bayesian posterior mastery p_{know} using confidence-stratified slip and guess probabilities ($p_{\text{slip}}[c], p_{\text{guess}}[c]$). The updated estimate is written synchronously to local `UserDefaults` and dispatched asynchronously to Neo4j via a background `PUT /quiz/mastery` request.
4. **Stateful Log Commit & Spaced Repetition:** When advancing to the next question, the client issues an authenticated `POST /quiz/questions/{uid}/log` request. The backend commits an immutable `:QUIZ_ANSWER` event linked to the active `:QuizSession` and updates the spaced-repetition schedule on the `(Student)-[:REVIEWING]->(Question)` edge.

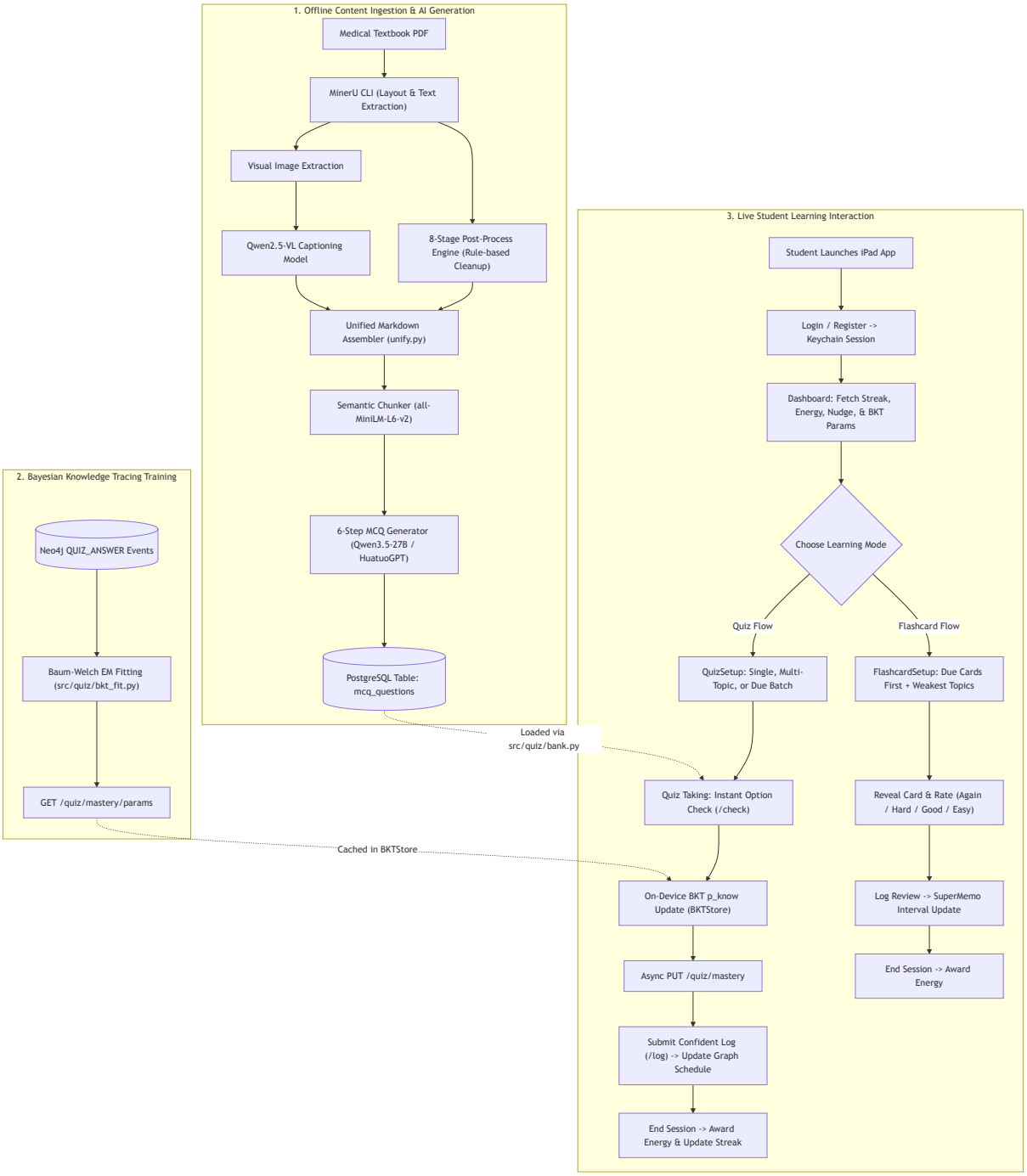
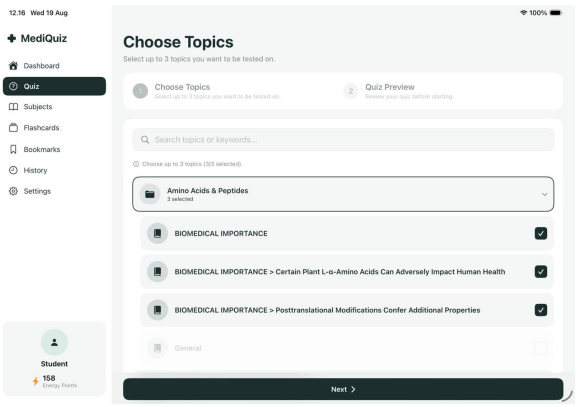
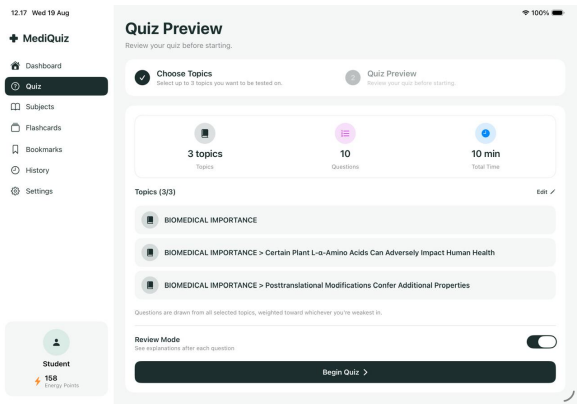


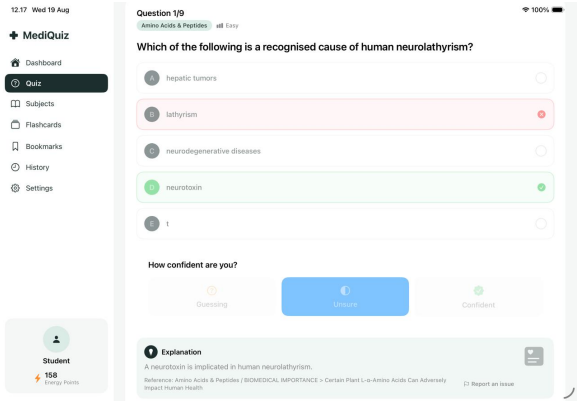
Figure 4: Complete system dataflow across the three subsystems: Offline Content Ingestion, BKT Parameter Training, and Live Student Learning Interaction.



(a) Multi-Topic Selection



(b) Adaptive Quiz Preview



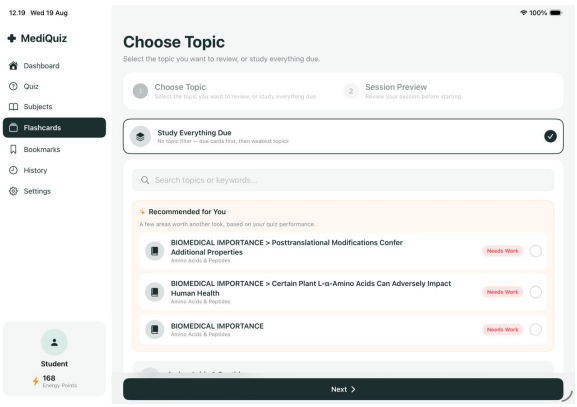
(c) Metacognitive Question Grading

Figure 5: Interactive Quiz Execution Lifecycle: (a) Multi-topic selection interface with real-time question counters, (b) Weakness-weighted quiz preview showing adaptive distribution across topics, and (c) Live question interface displaying instant option evaluation, 3-tier metacognitive confidence rating, and citation-grounded clinical explanation.

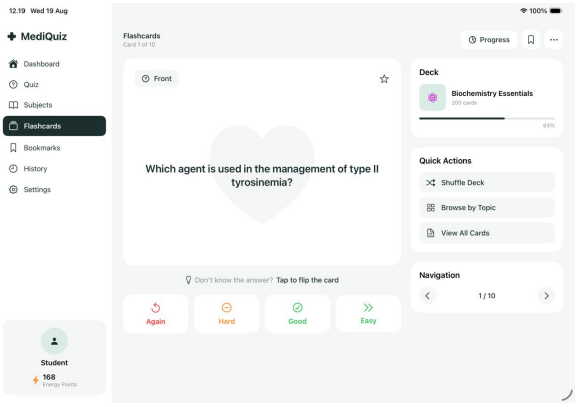
5. **Quiz Completion & Gamification:** `POST /quiz/sessions/{id}/end` finalises the session, aggregates score metrics, updates the student's daily activity streak, and awards +5 Energy currency in Neo4j.

Flow 4: Flashcard Spaced Repetition Lifecycle

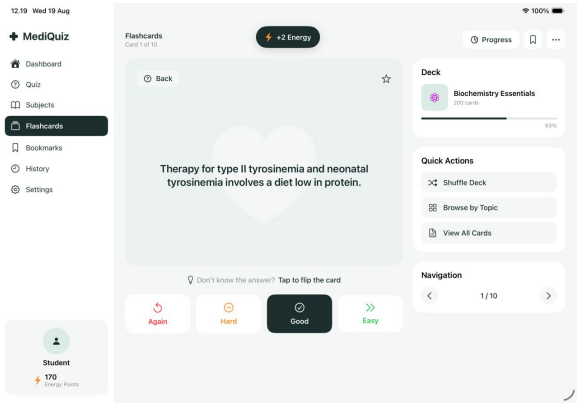
Flashcard study sessions employ a modified SuperMemo spaced repetition algorithm tailored to student knowledge state (illustrated in Figure 6):



(a) Weakness-Prioritized Setup



(b) Active Recall Prompt Front



(c) Answer Reveal & Rating Back

Figure 6: Flashcard Spaced Repetition Lifecycle: (a) Session configuration with "Study Everything Due" and targeted weak-topic recommendations, (b) Card front prompt displaying deck progress and navigation controls, and (c) Card back showing revealed clinical explanation, animated +2 Energy reward, and 4-tier SuperMemo recall rating buttons.

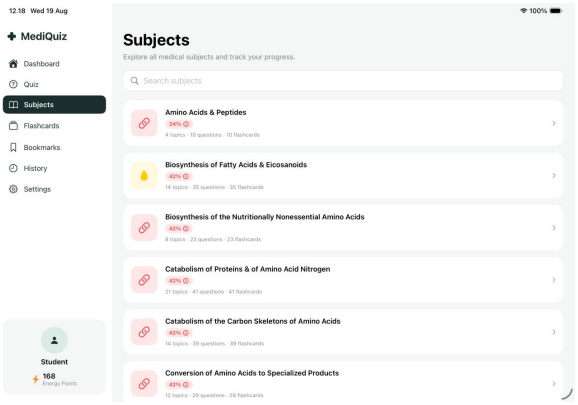
1. **Deck Assembly & Weakness Infill (Figure 6a):** When initiating a session via `POST /flashcards/sessions`, the backend first queries due cards where `next_review_at` $\leq$ now. Any remaining slots in the batch are dynamically infilled with flashcards corresponding to topics where the student exhibits low BKT mastery (p_{know}), surfaced under *Recommended for You* with "Needs Work" status tags.
2. **Active Recall Card Front (Figure 6b):** The client presents the question stem and prompt derived from triple synthesis, displaying current deck progress (e.g., Biochemistry Essentials, 63% progress) alongside deck navigation tools.
3. **Card Back Reveal & Energy Reward (Figure 6c):** Tapping "Tap to flip the card" or "Reveal Answer" dispatches `POST /flashcards/cards/{uid}/reveal` to retrieve the full

clinical answer text and explanation. An immediate visual reward animation awards +2 Energy points to the student's live balance.

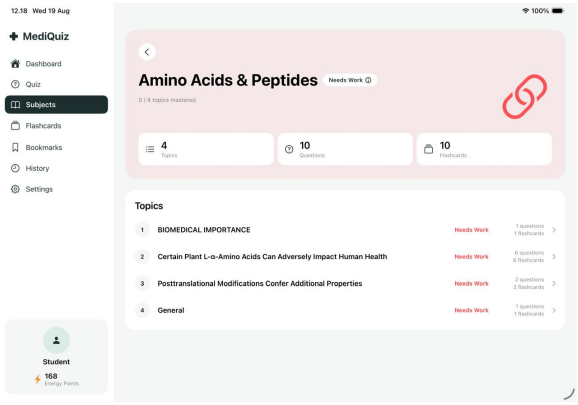
4. **Recall Rating & Interval Update (Figure 6c):** The student self-evaluates memory recall on a 4-point scale (*Again, Hard, Good, Easy*). `POST /flashcards/cards/{uid}/log` records an immutable `:FLASHCARD_REVIEW` node in Neo4j, recalculates interval days and repetition streaks on the `:Flashcard` node according to the SuperMemo multiplier rules, and commits the next review timestamp.

Flow 5: Adaptive Knowledge Tracing & Personalization Loop

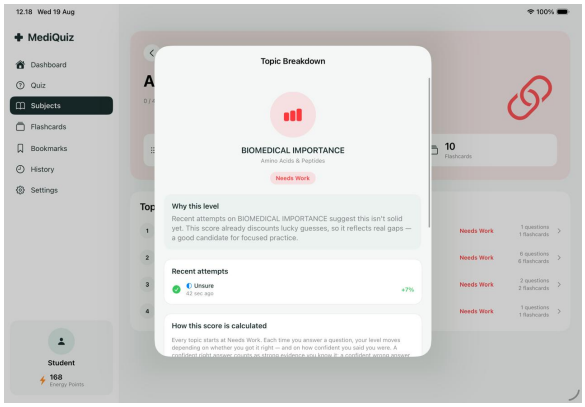
The personalization engine implements a closed-loop hybrid architecture combining server-side population fitting with client-side real-time state tracking (illustrated in Figure 7):



(a) Curriculum Subjects Browser



(b) Hierarchical Topic Breakdown



(c) Explainable BKT Diagnostic

Figure 7: Curriculum Navigation and Explainable Bayesian Mastery: (a) Subject overview browser displaying aggregate topic mastery percentages, (b) Subject detail view rendering subtopic mastery status badges, and (c) Explainable AI (XAI) diagnostic modal detailing confidence discounting, recent attempt trajectory, and Bayesian posterior updating.

1. **Server-Side Population Parameter Fitting (Periodic Batch):** Interaction sequences are extracted from pooled `:QUIZ_ANSWER` nodes in Neo4j. The Baum-Welch EM script (`src/quiz/bkt_fit.py`) executes Forward-Backward rescaling (E-step) and expectation maximisation (M-step) to derive global parameters: p_{init}, p_{transit}, and confidence-conditioned $p_{\text{slip}}(c)$ and $p_{\text{guess}}(c)$. Fitted parameters are cached and exposed via `GET /quiz/mastery/params`.

2. **Client-Side Real-Time Knowledge Tracing (iPadOS):** On application launch, `BKTStore` refreshes and caches population parameters. When the student answers a question with confidence c, `BKTStore` computes the closed-form Bayesian posterior:

$$p(L_t \mid y_t, c) = \frac{p(L_{t-1}) \cdot P(y_t \mid L_t, c)}{P(y_t \mid c)} \tag{1}$$

and applies the transition prior $p(L_{t+1}) = p(L_t \mid y_t, c) + (1 - p(L_t \mid y_t, c))p_{\text{transit}}$. The resulting p_{know} is stored in local storage and dispatched asynchronously to Neo4j via `PUT /quiz/mastery`.

3. **Curriculum Mastery Navigation (Figure 7a, 7b):** In `SubjectsView`, individual topic masteries are aggregated across medical subject modules (e.g., *Amino Acids & Peptides (34%)*, *Biosynthesis of Fatty Acids (42%)*). In `SubjectDetailView`, fine-grained subtopics display item counts and state labels (*Needs Work*, *Review Due*, *Mastered*) based on thresholded p_{know} values.

4. **Explainable AI (XAI) Diagnostic Modal (Figure 7c):** To demystify the machine learning model for learners, tapping the info badge opens the *Topic Breakdown* modal. This interface explains how the mastery level is calculated, illustrates recent attempt trajectories (e.g., *Correct · Unsure · +7%*), and explains why guessing is discounted: *"Recent attempts suggest this isn't solid yet. This score already discounts lucky guesses, so it reflects real gaps—a good candidate for focused practice."*

5. **Downstream Personalization Delivery:**

   - *Weighted Quiz Selection:* Multi-topic quizzes sample questions with weights inversely proportional to topic mastery ($\propto 1 - p_{\text{know}}$).
   - *Flashcard Weak-Topic Infill:* Practice decks prioritize review cards from topics exhibiting the lowest mastery scores.
   - *Adaptive Nudges & Progress:* The backend `GET /students/me/nudge` endpoint inspects `:MASTERS` edges to surface high-priority study recommendations on the dashboard.

Flow 6: Offline Ingestion & Graph-Grounded MCQ Generation Pipeline

The offline ingestion pipeline converts unstructured medical textbooks into formal knowledge graphs and verified multiple-choice questions across five sequential stages:

1. **Stage 1: Parsing & Extraction:** The MinerU CLI and notebook (`notebooks/parse_books.ipynb`) process raw textbook PDFs to extract layout bounding boxes (`_middle.json`), content streams (`_content_list.json`), and embedded anatomical figures.

2. **Stage 2: Deterministic Book Cleaning (`src/post_process_book`):** Cleans MinerU output by stripping image blocks, headers/footers, and publisher citation parentheticals while preserving paragraph body text, list structures, tables, mathematical formulas, and inline figure mentions. Outputs `<chapter>.cleaned.md` and `<chapter>.page_map.json`.

3. **Stage 3: Structural Document Rewriting (`document-rewriter`):** Under strict faithfulness constraints, carries over textbook prose verbatim into structured XML-like study markdown (`<doc>`, `<con>`, `<fig>`, `<tbl>`), resolving in-document cross-references into `<figref>`/`<tblref>` and calling HuatuoGPT-o1-72B exclusively for the document `<topic>` line.

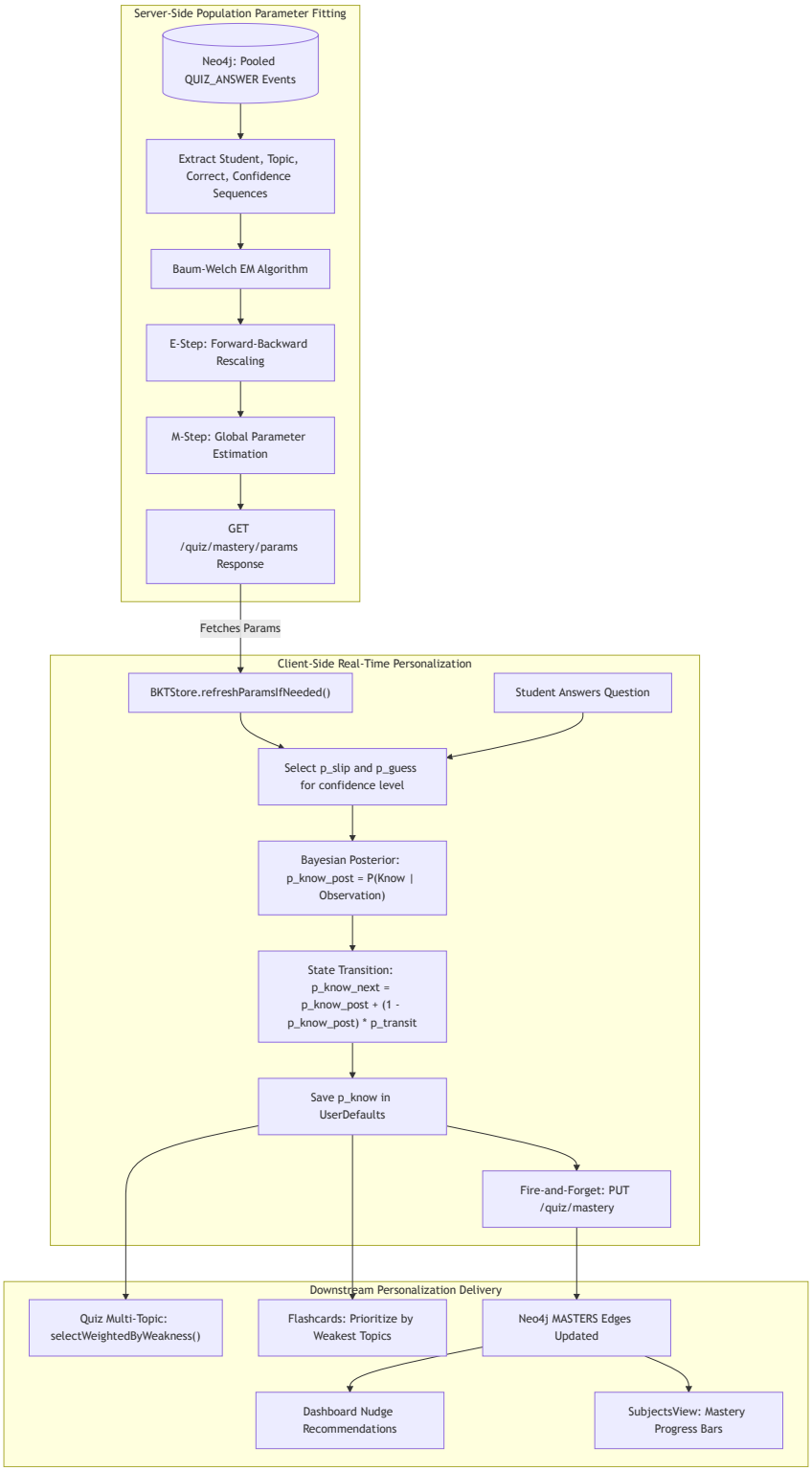


Figure 8: Hybrid BKT personalization loop: server-side Baum-Welch EM parameter fitting and client-side real-time knowledge tracing on iPadOS.

4. **Stage 4: Domain Knowledge Graph Construction (`knowledge-graph-pipeline`):** The 8-stage `medkg` engine executes scispaCy NER, SapBERT entity linking with UMLS semantic type label correction, HuatuoGPT-o1-72B relation and modifier extraction, 14 deterministic relation guards, post-coordination modifier minting, RDF quad assertion, and OWL-RL reasoning.

5. **Stage 5: Graph-Grounded MCQ Generation & Database Population (`medkg/mcq.py`):** Synthesizes single-best-answer MCQs directly from asserted semantic triples across 13 core clinical predicate templates. Enforces 3-tier distractor ranking with open-world multi-graph blocklisting and batch LLM stem phrasing (`phrase_items`) with deterministic template fallback. Validated questions are committed to PostgreSQL (`mcq_questions`).

Flow 7: Domain Knowledge Graph (medkg) Pipeline

The domain knowledge graph pipeline constructs a formal, ontology-grounded biomedical semantic graph across eight stages:

1. **Stage 1 (Parse):** Ingests semantically tagged Markdown containing clinical inline annotations (`<con>`, `<def>`, `<clin>`), validating character spans and emitting normalized text alongside offset-anchored IR chunks.

2. **Stage 2 & 3 (NER, Entity Linking & Relation Extraction):** Executes scispaCy NER (`en_ner_bc5cdr_md`, `en_ner_bionlp13cg_md`) and SapBERT dense entity linking via FAISS ($\cos \geq 0.70$). UMLS semantic types from `semantic_types.json` dynamically re-derive span ontology labels to prevent domain classification errors before HuatuoGPT-o1-72B extracts 27-relation medical triples and qualifiers.

3. **Guard Layer (14 Deterministic Guards):** Audits all extracted candidates against directional connectives, governed roles, and evidence scope, demoting suspect relations to `polarity = "uncertain"`.

4. **Stage 4 (Post-coordination):** Mints scoped `:INSTANCE` nodes to model complex clinical modifiers (especially `deviation` modifiers such as deficiency or excess on substances) to preserve clinical semantics without combinatorial ontology explosion.

5. **Stage 5 (Multimodal Grounding):** Handles figure depiction structures, strictly firewalling visual descriptions so that figure captions never become ungrounded clinical facts.

6. **Stage 6 (RDF Assertion):** Serializes extracted assertions into W3C RDF Named Quads (`<subject> <predicate> <object> <graph>`) partitioned per document and section, with provenance metadata.

7. **Stage 7 (OWL-RL Reasoning):** Applies deterministic OWL 2 RL inference rules in-process via `owlrl` to materialize implicit taxonomic, inverse, and transitive relationships into `urn:graph:inferred`.

8. **Stage 8 (Serving & Subsetting):** Exports the reasoned knowledge base as N-Quads, hosts a SPARQL query endpoint, and supports document/section graph subsetting for downstream MCQ generation and semantic exploration.

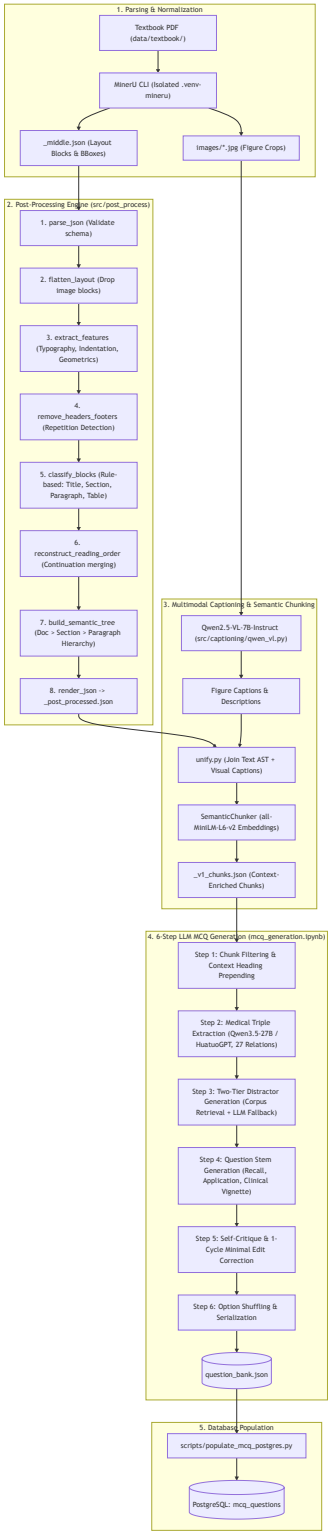


Figure 9: 5-stage offline ingestion pipeline: PDF parsing, deterministic cleaning, structural document rewriting, domain KG construction, and graph-grounded MCQ generation.

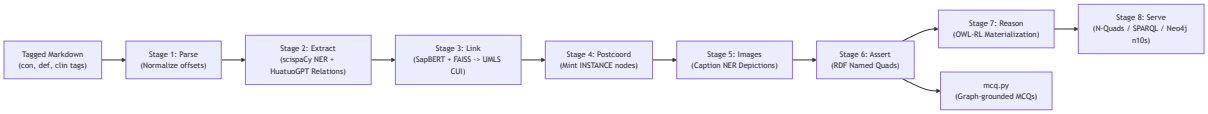


Figure 10: 8-stage medkg pipeline: Parse → NER → Entity Linking → Post-coordination → RDF Assertion → OWL-RL Reasoning → Serve (N-Quads / SPARQL / Subsets).

Model Deployment Method

Model Zoo & Task Allocation Matrix

Model	Type	Task & Responsibility	Runtime / Env
HuatuoGPT-o1-72B (4-bit)	Medical LLM (Reasoning)	Document tagging & 27-relation biomedical triple extraction	vLLM OpenAI-compatible endpoint
Qwen3.5-27B	General / Medical LLM	MCQ stem phrasing, stylistic rephrasing & validation guardrails	vLLM / Local HuggingFace
Qwen2.5-VL-7B-Instruct	Vision-Language Model	Medical figure captioning & diagram transcription	PyTorch / Accelerate (MPS / CUDA)
SapBERT (`cambridgeltl`)	Biomedical Sentence Transformer	Entity linking to UMLS/SNOMED CT concepts via FAISS ($\cos \geq 0.70$)	FAISS `IndexFlatIP` (CPU)
MedGemma 4B + UNet	Multimodal VLM + CNN	Diagram classifier & leader line endpoint detector	`mlx-community` / PyTorch
Baum-Welch EM Engine	2-State HMM	Global parameter fitting for Knowledge Tracing	NumPy / SciPy (Pure CPU)
On-Device BKT Engine	2-State HMM Forward Updater	Real-time student mastery calculation (p_{know}) on iPadOS	Swift Standard Library (Zero Network)

Deployment Topologies & Inference Infrastructure

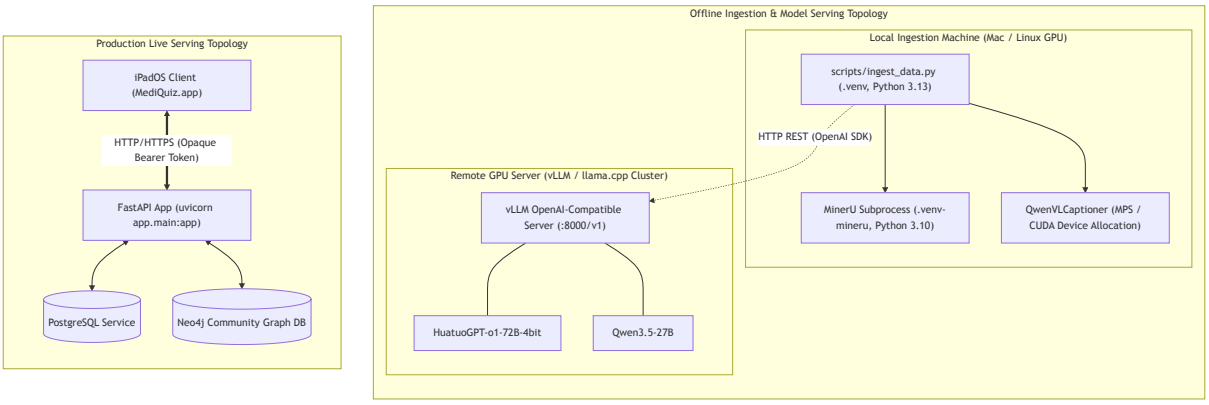


Figure 11: Deployment topology: Offline Ingestion infrastructure (local Mac / remote GPU cluster) and Production Live Serving infrastructure (FastAPI + PostgreSQL + Neo4j).

Zero-Inference Live Serving Architecture

A critical architectural achievement is that **the live FastAPI serving application performs zero model inference at runtime**:

1. **CPU-Only Bound:** The production server requires only standard CPU instances (e.g. 2 vCPUs, 4 GB RAM) with negligible footprint.
2. **Instant Latency:** Response times for `/quiz/questions`, `/quiz/check`, and `/quiz/log` remain sub-15 ms because they execute indexed PostgreSQL queries and atomic Cypher graph transactions without model queue delays.
3. **High Reliability:** Server stability is completely decoupled from GPU drivers, CUDA version mismatches, or VRAM exhaustion.

Multi-Environment Isolation Strategy

Because several specialised medical AI libraries carry incompatible Python and C-extension dependencies, the project isolates runtimes into three dedicated virtual environments:

- `.venv/` (Live Backend): Python ≥ 3.13 (managed by `uv`). Dependencies: FastAPI, Uvicorn, Neo4j, SQLAlchemy, PyTorch, Transformers, Accelerate, Pydantic, SlowAPI.
- `.venv-mineru/` (MinerU Parser): Python 3.10/3.11. Dependencies: `mineru`, `magic-pdf`, Detectron2, PaddlePaddle.
- `knowledge-graph-pipeline` environment: Python 3.10/3.11. Dependencies: scispaCy, spacy < 3.8, faiss-cpu, rdflib, owlrl. Required for Stage 2 NER, Stage 3 UMLS linking, and Stage 7 reasoning.

Hardware Optimisation & Native Runtime Guards

- **Dynamic Device Selection with MPS Bug Workarounds** (`src/captioning/qwen_vl.py`): Dynamically allocates execution targets (CUDA $\rightarrow$ MPS $\rightarrow$ CPU) and implements explicit memory purging (`torch.mps.empty_cache()`, `gc.collect()`) upon completion to operate smoothly within Apple Silicon Unified Memory constraints.
- **Native macOS Segfault Protection (`knowledge-graph-pipeline/medkg/native_env.py`):** Addresses an issue on macOS where PyTorch, Thinc/Blis (spaCy), and FAISS clash when initialising separate OpenMP thread pools, resulting in `Segmentation fault: 11`. Setting `MEDKG_NATIVE_THREADS=1` and `KMP_DUPLICATE_LIB_OK=TRUE` at the top of the package before any C-extensions load prevents segmentation crashes.

Source Code Structure

Repository 1: Backend & Ingestion Preprocessing

(`intelligent-tutoring-system-for-medical-student`)

```
intelligent-tutoring-system-for-medical-student/
├── app/                        # FastAPI HTTP Layer & Routers
│   ├── main.py                 # Entry point, middleware, CORS, routers
│   ├── auth_middleware.py      # SessionAuthMiddleware: Allowlist & token resolver
│   ├── dependencies.py         # Neo4j driver injection & get_current_student_id
│   ├── errors.py               # Unified error envelope & global exception handlers
│   ├── limiter.py              # SlowAPI rate limiter configuration
│   ├── openapi.py              # OpenAPI schema generator & error response docs
```

```
│   ├── schemas.py              # Pydantic DTO request/response contracts
│   └── routers/
│       ├── students.py         # Auth, profile, streak, energy, nudge endpoints
│       ├── quiz.py             # Topics, questions, check/log, sessions, mastery
│       └── flashcards.py       # Cards, reviews, sessions, due-review queue
│
├── src/                        # Core Business Logic & Domain Services
│   ├── student_kg/             # Neo4j Student Knowledge Graph Drivers
│   ├── quiz/                   # Quiz Subsystem & Baum-Welch EM BKT Fitter
│   ├── flashcards/             # Flashcard Subsystem & SuperMemo Scheduler
│   ├── captioning/             # Qwen2.5-VL-7B vision-language captioner
│   ├── diagram_processing/     # MedGemma + U-Net leader line detection
│   └── post_process_book/      # Deterministic MinerU book cleaner & page mapper
│
├── notebooks/                  # Exploratory & Parsing Notebooks
│   ├── parse_books.ipynb       # MinerU PDF extraction notebook
│   └── knowledge_tracing.ipynb  # BKT validation & Baum-Welch EM research
│
├── scripts/                    # CLI Seeding & PostgreSQL loader scripts
│   └── populate_mcq_postgres.py  # Loads validated question JSON into PostgreSQL
└── docker-compose.yml          # Local infrastructure (Postgres, Neo4j, MinIO)
```

Repository 2: Structural Document Rewriter

(document-rewriter)

```
document-rewriter/
├── rewrite_medical_md.py       # Faithful textbook prose tagging (<doc>, <con>, <fig>)
├── TAG_SCHEMA.txt              # Output XML-like tag vocabulary documentation
└── batch_process.sh            # Batch rewriter driver script
```

Repository 3: Domain Knowledge Graph & MCQ Pipeline

(knowledge-graph-pipeline)

```
knowledge-graph-pipeline/
├── medkg/                      # Core medkg library
│   ├── stage1_parse.py         # Normalised text & structured IR chunking
│   ├── stage2_extract.py       # scispaCy NER & HuatuoGPT-o1 triple extraction
│   ├── stage3_link.py          # SapBERT/FAISS linking & UMLS semantic typing
│   ├── stage4_postcoord.py     # Post-coordination modifier minting (:INSTANCE)
│   ├── stage6_assert.py        # RDF Named Quads serialization
│   ├── stage7_reason.py        # OWL-RL in-process inference (owlrl)
│   ├── stage8_serve.py         # SPARQL & N-Quads dataset query/export
│   └── mcq.py                  # Graph-grounded single-best-answer MCQ synthesis
├── run.py                      # Pipeline runner (single-document / corpus mode)
├── query.py                    # SPARQL exploration CLI
└── subset.py                   # Graph subsetting utility
```

Repository 4: Frontend Client

(personalized-learning-ios)

```
personalized-learning-ios/personalized-medical-learning-ios/
├── App/                        # App Lifecycle & Root Navigation
│   ├── ...App.swift            # @main entry point
│   ├── ContentView.swift       # Auth routing: RootView or LoginView
│   └── RootView.swift          # Sidebar navigation container & active view switch
```

```
├── Core/                       # Shared Infrastructure & Foundations
│   ├── Networking/             # APIClientProtocol & URLSession client
│   ├── Persistence/            # KeychainStore, SessionManager, BKTStore
│   └── UI/                     # SidebarView, TopBarView
│
└── Features/                   # Domain Feature Modules (MVVM)
    ├── Auth/                   # LoginView, RegisterView, ViewModels
    ├── Dashboard/              # DashboardView, Streak/Nudge ViewModels
    ├── Quiz/                   # QuizView, QuizSetupView, QuizViewModel
    ├── Flashcard/              # FlashcardView, FlashcardSetupView
    ├── Mastery/                # MasteryAPI, BKT parameter sync
    ├── Subjects/               # SubjectsView, SubjectDetailView
    ├── History/                # HistoryView, HistoryViewModel
    ├── Bookmark/               # BookmarkView
    └── Settings/               # SettingsView (backend endpoint, logout)
```

Cross-Repository API & Protocol Contract Mapping

HTTP Route	Method	Auth	Primary Database Action
`/students/register`	POST	None	Creates `Student` & genesis `ENROLLMENT` in Neo4j
`/students/login`	POST	None	Creates `:Session` node in Neo4j
`/students/logout`	POST	Bearer	Sets `Session.revoked_at` in Neo4j
`/students/me/profile`	GET	Bearer	Reads `Student` node from Neo4j
`/students/me/streak`	GET	Bearer	Evaluates UTC activity dates from Neo4j
`/students/me/streak/restore`	POST	Bearer	Spends 10 Energy to bridge 1 missed day
`/students/me/energy`	GET	Bearer	Reads `Student.energy` from Neo4j
`/students/me/nudge`	GET	Bearer	Finds soonest due quiz & flashcard items
`/quiz/topics`	GET	None	Reads distinct `topic_tag` from PostgreSQL
`/quiz/topics/{path}/questions`	GET	None	Reads sanitised questions from PostgreSQL
`/quiz/questions/{uid}/check`	POST	None	In-memory answer verification (Stateless)
`/quiz/topics/{path}/sessions`	POST	Bearer	Creates `QuizSession` node in Neo4j
`/quiz/questions/{uid}/log`	POST	Bearer	Creates `QUIZ_ANSWER` & updates `REVIEWING` edge
`/quiz/sessions/{id}/end`	POST	Bearer	Aggregates score, awards Energy in Neo4j
`/quiz/review/due`	GET	Bearer	Queries `REVIEWING` edges where `next_review_at` $\leq$ now
`/quiz/mastery`	PUT	Bearer	Overwrites `:MASTERS` edge `p_know` in Neo4j
`/quiz/mastery/params`	GET	Bearer	Runs / caches Baum-Welch EM fit on pooled events
`/flashcards/cards`	GET	None	Projects questions to flashcard models
`/flashcards/cards/{uid}/reveal`	POST	None	Returns card back text & explanation

HTTP Route	Method	Auth	Primary Database Action
`/flashcards/sessions`	POST	Bearer	Creates `FlashcardSession` in Neo4j
`/flashcards/cards/{uid}/log`	POST	Bearer	Creates `FLASHCARD_REVIEW` & updates `Flashcard` node
`/flashcards/sessions/{id}/end`	POST	Bearer	Marks `FlashcardSession.status = 'completed'`
`/flashcards/review/due`	GET	Bearer	Queries `Flashcard` nodes where `next_review_at` $\leq$ now

Mathematical Foundations & Algorithms

Bayesian Knowledge Tracing (BKT) with Confidence Emissions

Standard BKT models a student's latent knowledge of a specific medical topic as a 2-state Hidden Markov Model:

- **State 0:** *Doesn't Know*
- **State 1:** *Knows*

To account for metacognitive awareness, emission probabilities p_{slip} and p_{guess} are conditioned on the student's self-reported confidence $c \in \{\text{confident, unsure, guessing}\}$:

Confidence (c)	p_{slip}	p_{guess}	**Pedagogical Interpretation**
Confident	0.05	0.10	Confident-wrong strongly penalises p_{know} as a misconception.
Unsure	0.10	0.25	Standard baseline BKT behaviour.
Guessing	0.30	0.50	Correct guess does not falsely award mastery.

Observation Update (Bayes Rule): Given prior mastery p_{know} and observation $O_t \in \{\text{Correct, Incorrect}\}$ with confidence c:

$$O_t = \text{Correct}: \quad p_{\text{know}}^{\text{post}} = \frac{p_{\text{know}} \cdot (1 - p_{\text{slip}}[c])}{p_{\text{know}} \cdot (1 - p_{\text{slip}}[c]) + (1 - p_{\text{know}}) \cdot p_{\text{guess}}[c]}$$

$$O_t = \text{Incorrect}: \quad p_{\text{know}}^{\text{post}} = \frac{p_{\text{know}} \cdot p_{\text{slip}}[c]}{p_{\text{know}} \cdot p_{\text{slip}}[c] + (1 - p_{\text{know}}) \cdot (1 - p_{\text{guess}}[c])}$$

Learning State Transition (accounting for knowledge acquisition during the attempt):

$$p_{\text{know}}^{\text{next}} = p_{\text{know}}^{\text{post}} + (1 - p_{\text{know}}^{\text{post}}) \cdot p_{\text{transit}}$$

Baum-Welch Expectation-Maximisation (EM) Parameter Fitting

Implemented in `src/quiz/bkt_fit.py`, the Baum-Welch EM algorithm fits optimal global parameters across all pooled student sequences:

1. **E-Step (Forward-Backward Rescaling):** Compute smoothed posterior state probabilities $\gamma_t(i) = P(S_t = i \mid \mathbf{O})$ and transition probabilities $\xi_t(0, 1) = P(S_t = 0,\, S_{t+1} = 1 \mid \mathbf{O})$.

2. **M-Step (Closed-Form Aggregation):**

$$p_{\text{init}} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \gamma_0^{(s)}(1) \qquad p_{\text{transit}} = \frac{\sum_s \sum_t \xi_t^{(s)}(0,1)}{\sum_s \sum_t \gamma_t^{(s)}(0)}$$

$$\forall c: \; p_{\text{slip}}[c] = 1 - \frac{\sum_s \sum_{t:\, O_t = \text{Correct},\, C_t = c} \gamma_t^{(s)}(1)}{\sum_s \sum_{t:\, C_t = c} \gamma_t^{(s)}(1)} \qquad p_{\text{guess}}[c] = \frac{\sum_s \sum_{t:\, O_t = \text{Correct},\, C_t = c} \gamma_t^{(s)}(0)}{\sum_s \sum_{t:\, C_t = c} \gamma_t^{(s)}(0)}$$

Dual-Track Spaced Repetition Scheduling

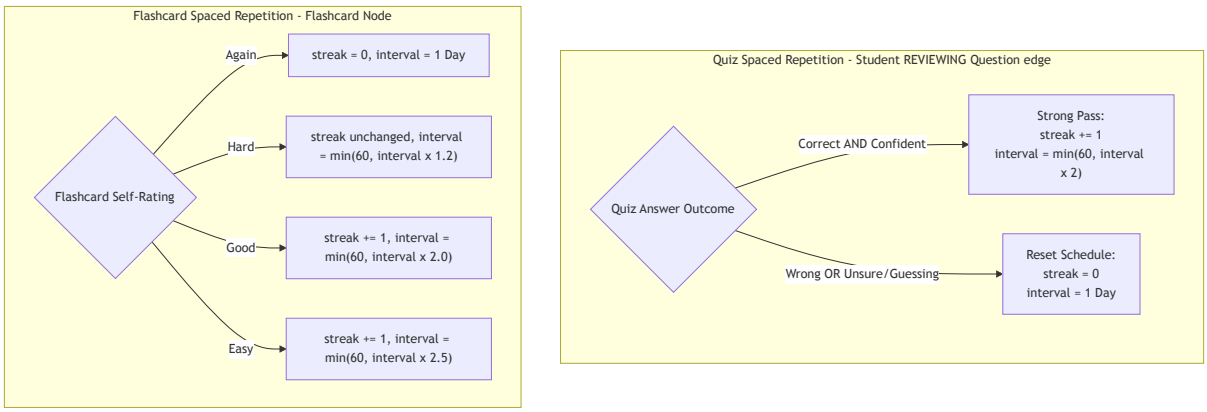


Figure 12: Dual-track spaced repetition: Quiz track (REVIEWING edge) and Flashcard track (SuperMemo self-rating intervals).

The two tracks use distinct scheduling mechanics:

- **Quiz Track** (`(Student)-[:REVIEWING]->(Question)` edge): Correct + Confident ⇒ `streak += 1`, `interval = min(60, interval×2)`. Wrong or low-confidence ⇒ reset: `streak = 0`, `interval = 1`.
- **Flashcard Track** (`Flashcard` node): *Again* ⇒ reset to 1 day; *Hard* ⇒ interval × 1.2; *Good* ⇒ streak +1, interval × 2.0; *Easy* ⇒ streak +1, interval × 2.5. All capped at 60 days.

Gamification Economy: Streaks, Energy & Nudge Heuristics

The student learning experience is orchestrated through a gamified habit-loop on the iPadOS dashboard (Figure 13), connecting backend persistence in Neo4j to cognitive reinforcement heuristics:

1. **Review Nudge Recommendation Engine (`GET /students/me/nudge`):** Aggregates total due quiz questions and due flashcards across the student's spaced repetition schedules (e.g., *"54 due for review: 28 quiz questions and 26 flashcards"*), identifying the single item with min(`next_review_at`) across both modes to populate the prominent top dashboard reminder banner with a direct one-tap review action.

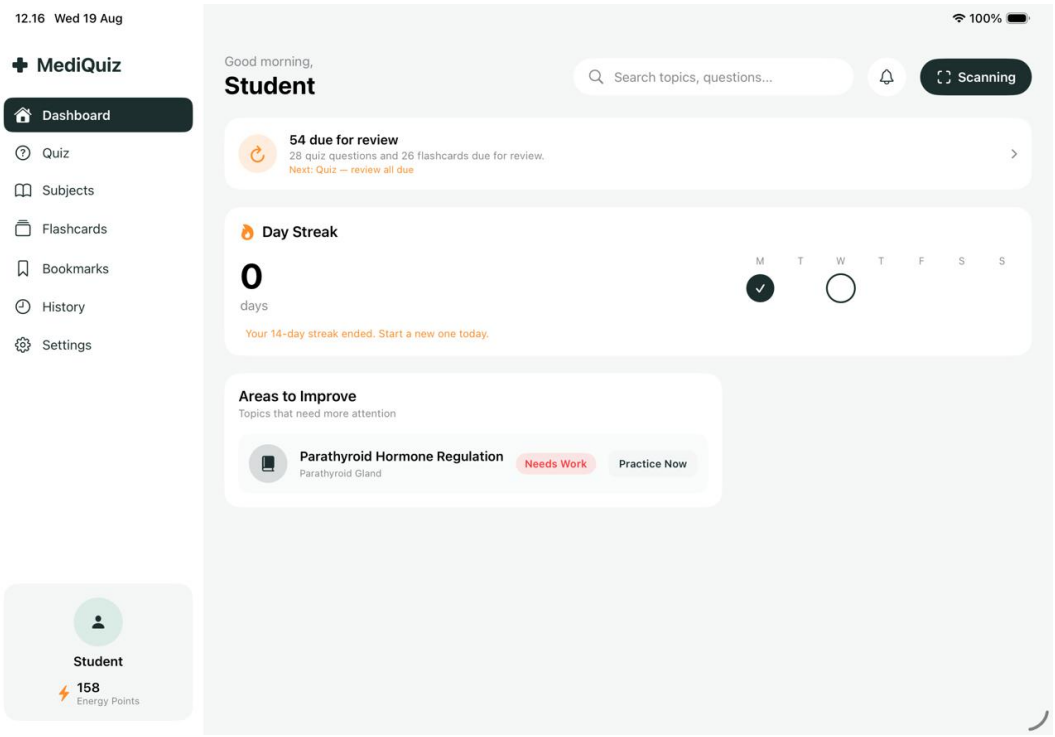


Figure 13: MediQuiz Student Dashboard: Integrating review nudge heuristics (due quiz questions and flashcards), active day streak tracking with recovery prompts, targeted weak-topic remediation cards, and live energy points ledger.

2. **Consecutive Streak Tracking (`src/student_kg/streak.py`):** Evaluates distinct calendar days in UTC where the student generated at least one `QUIZ_ANSWER` or `FLASHCARD_REVIEW`. A gap ≥ 2 days resets `current_streak = 0` while `previous_streak` retains the broken length, displaying an encouraging prompt (e.g., *"Your 14-day streak ended. Start a new one today"*) alongside a weekly completion calendar.

3. **Streak Restoration (`POST /students/me/streak/restore`):** If the student missed exactly 1 day, they spend **10 Energy** units to bridge the gap and restore their previous streak.

4. **Energy Points Ledger (`src/student_kg/energy.py`):** Earned: +5 per Quiz Session completion, +1 to +2 per Flashcard Review. Spent: −10 per Streak Restore. The student's live balance (e.g., 158 Energy Points) is permanently visible on the sidebar profile widget.

5. **Targeted Weakness Remediation ("Areas to Improve"):** The dashboard queries the student's lowest p_{know} topics from Neo4j (`:MASTERS` edges) to display high-impact practice cards (e.g., *Parathyroid Hormone Regulation — Needs Work*) with a direct *Practice Now* CTA button.

Responsible AI Integration

MediQuiz applies responsible AI principles across three dimensions: user data sovereignty, content licensing integrity, and transparency in algorithmic decision-making. Each principle is grounded in concrete system design choices rather than policy statements alone.

On-Device Knowledge State

The student's primary knowledge representation — the Bayesian posterior mastery estimate p_{know} and its underlying attempt history — is computed and persisted entirely on the student's own iPad. The BKT update rule executes within `BKTStore` in the Swift Standard Library with zero network dependency; the result is written synchronously to local `UserDefaults` before any background sync is dispatched to Neo4j. This means the student retains direct, local custody of their most sensitive learning signal at all times. No third-party inference endpoint, cloud ML service, or external model host ever processes individual mastery trajectories. The server receives only the final scalar p_{know} value — not the raw answer sequence used to derive it — limiting the exposure of detailed performance data to the minimum necessary for aggregate parameter fitting.

Authorised Content Sources & Institutional Scope

All educational content ingested by MediQuiz is sourced exclusively from AccessMedicine, a licensed medical reference platform whose terms of use are complied with throughout the pipeline. The ingestion workflow applies strict faithfulness constraints: `document-rewriter` carries over textbook prose verbatim under a "no paraphrasing" instruction, and the offline pipeline is operated on institution-controlled infrastructure rather than third-party cloud services. Critically, the application is not publicly distributed; it is deployed solely within the university environment for enrolled medical students in Ciputra University, ensuring that the use of licensed materials remains within the permitted institutional scope. The question bank produced from these sources is stored in a private PostgreSQL instance and is not exposed to external systems or users.

Explainable Student Mastery & Bounded Scope

Learners are shown the confidence estimate behind each mastery label — not a binary pass/fail verdict. The *Topic Breakdown* modal (Figure 7c) surfaces the Bayesian posterior value, the recent attempt trajectory with per-attempt confidence weighting, and a plain-language explanation of why the estimate is discounted for lucky guesses (e.g., *"This score already discounts lucky guesses, so it reflects real gaps"*). The scope of explainability is deliberately bounded to the learning-progress domain: BKT explains skill-tracking logic only. The system makes no clinical diagnostic claims, offers no medical advice, and produces no patient-facing output. Every explanation text is derived from verified textbook provenance rather than generative LLM inference at serve time, preventing hallucination of clinical facts during live student interaction.

Operations, Runbook & Local Setup

Backend & Infrastructure Bootstrapping

Prerequisites: macOS or Linux, `uv` package manager, Docker & Docker Compose.

1. **Setup Environment & Databases:**

```
# Navigate to backend repo
cd intelligent-tutoring-system-for-medical-student

# Configure environment variables
cp .env.example .env
```

```
# Start Neo4j, PostgreSQL, and MinIO containers
docker compose up -d postgres neo4j minio

# Synchronize Python dependencies via uv
uv sync
```

2. **Populate Seed Data:**

```
# Populate PostgreSQL mcq_questions table
uv run python scripts/populate_mcq_postgres.py

# (Optional) Populate synthetic student interactions
uv run python scripts/generate_dummy_interactions.py
```

3. **Launch Live FastAPI Server:**

```
uv run uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Verify: `curl http://localhost:8000/health` → `{"status":"ok"}`. Interactive Swagger docs: `http://localhost:8000/docs`.

iPadOS Client Setup & Compilation

Prerequisites: Xcode 16.0+ (iPadOS 26.5+ SDK); Simulator or Physical iPad with Developer Mode enabled.

1. **Configure Host IP** in `Core/Networking/APIConfig.swift`:

```
enum APIConfig {
    // Set to local development machine IP on LAN
    static let baseURL = URL(string: "http://10.67.52.231:8000")!
}
```

2. **Open Project & Run:**

```
cd personalized-learning-ios
open personalized-medical-learning-ios.xcodeproj
```

Select an iPad simulator (e.g. iPad Pro 13-inch) in **Landscape Orientation** and press `Cmd + R`.

Dataset Ingestion & Content Regeneration

To process new textbook PDFs through the multimodal pipeline:

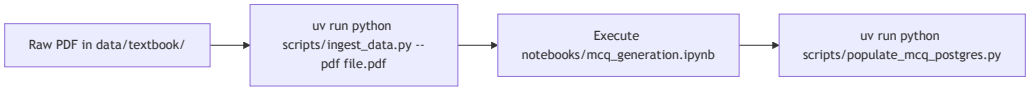


Figure 14: Dataset ingestion runbook: PDF → ingest script → MCQ notebook → PostgreSQL population.

```
# 1. Parse PDF via MinerU and clean using post_process_book
cd intelligent-tutoring-system-for-medical-student
python -m src.post_process_book.run_cleaner path/to/output_middle.json -o output/

# 2. Re-emit structurally tagged study markdown via document-rewriter
cd ../document-rewriter
python rewrite_medical_md.py path/to/chapter.cleaned.md -o output/chapter.tagged.md

# 3. Construct biomedical knowledge graph via knowledge-graph-pipeline
cd ../knowledge-graph-pipeline
python run.py --input path/to/chapter.tagged.md

# 4. Generate graph-grounded MCQs from asserted triples
python mcq.py artifacts/docs/chapter/ir.json --out questions.json

# 5. Populate live PostgreSQL question bank
cd ../intelligent-tutoring-system-for-medical-student
uv run python scripts/populate_mcq_postgres.py
```

Verification & Automated Test Suite

```
# Run backend pytest suite
cd intelligent-tutoring-system-for-medical-student
uv run pytest -v
```

Developer Extension Guide & Future Roadmap

Recipe: Adding a New Backend Endpoint

To expose a new API endpoint (e.g. `POST /students/me/bookmarks`):

1. **Define Schema:** Add request/response Pydantic models to `app/schemas.py`.
2. **Implement Domain Logic:** Add graph query functions in `src/student_kg/`.
3. **Register Router Route:** Add a route handler to `app/routers/students.py` specifying `@router.post(...)`.
4. **Configure Auth Allowlist (If Public):** If unauthenticated, update regex rules in `app/auth_middleware.py`.
5. **Add Tests:** Create corresponding test cases in `tests/`.

Recipe: Adding a New iPadOS Feature / View

To implement a new feature view (e.g. `BookmarksView`):

1. **Declare Protocol Method:** Add the `async` API signature to `Core/Networking/APIClientProtocol.swift`.

2. **Implement Network Call:** Implement the method in `Features/Bookmark/BookmarkAPI.swift` using `client.get(...)` or `client.send(...)`.

3. **Build ViewModel:** Create `BookmarkViewModel` conforming to `ObservableObject` marked `@MainActor`, holding `@Published` state and injecting `any APIClientProtocol`.

4. **Construct SwiftUI View:** Build the interface in `BookmarkView.swift` observing `BookmarkViewModel`.

5. **Register Navigation:** Add an enum case to `SidebarItem` in `Core/UI/SidebarView.swift` and wire the switch statement in `App/RootView.swift`.

Limitations and Next Steps

Limitations

- **Absence of Systematic Validation for Knowledge Graph, Retrieval, and Question Generation:** While the pipeline enforces structural schema validation, deterministic relation guards, and open-world distractor safety guardrails, the system has not undergone comprehensive quantitative evaluation or expert clinical validation. Specifically: (1) the medical knowledge graph construction (triple extraction via HuatuoGPT-o1-72B and SapBERT entity linking) lacks formal precision/recall benchmarking and expert medical ontology audit against gold-standard annotated corpora; (2) the GraphRAG and semantic retrieval mechanisms have not been quantitatively benchmarked using standard information retrieval metrics (e.g., Hit@K, MRR, Context Relevance, and Groundedness); and (3) the generated multiple-choice questions (MCQs) have not yet been evaluated through systematic clinical faculty review or psychometric item analysis to verify clinical accuracy, nuance, and absence of subtle medical hallucinations prior to deployment.
- **Absence of Bahasa Indonesia Localization (Monolingual English Pipeline):** The offline knowledge extraction and question generation pipelines (utilising HuatuoGPT-o1-72B and Qwen3.5-27B) as well as the client user interface operate exclusively in English. Consequently, the platform cannot directly ingest Indonesian-language medical textbooks or support students who prefer native-language instructional materials and localized clinical terminology.
- **Incomplete End-to-End Multimodal Question Generation:** Medical education relies heavily on visual comprehension, including anatomical diagrams, histological micrographs, diagnostic imaging, and biochemical pathways. Although core multimodal preprocessing modules have been successfully developed—including document figure and layout parsing, vision-language figure captioning, and diagram leader-line detection with OCR-based text label replacement (masking text labels with numbers for anatomical identification)—these processed visual assets have not yet been connected to an automated question generation pipeline. Consequently, the MCQ generation engine currently operates solely on textual semantic triples, and cannot yet formulate visual questions or multimodal clinical case scenarios grounded directly in the processed diagrams and charts.
- **Cold-Start Latency in Bayesian Knowledge Tracing (BKT):** The student personalization engine leverages Bayesian Knowledge Tracing to infer latent mastery probabilities across granular medical concepts. Because the model relies on historical practice trajectories, newly enrolled students lack sufficient interaction records. As a result, initial mastery estimations default to uncalibrated global priors ($P(L_0)$), leading to suboptimal question recommendations and difficulty matching until a critical mass of formative responses is gathered.

Next Steps

- **Execute Pilot Testing with Student Cohorts for Knowledge Tracing Calibration:** Deploy the iPadOS application to small cohorts of medical students in controlled pilot trials to validate application usability and stability, while systematically gathering real-world interaction logs (response correctness, latencies, and confidence ratings). This empirical data will serve to calibrate Bayesian Knowledge Tracing (BKT) parameters (p_{init}, p_{transit}, p_{slip}, p_{guess}) and alleviate cold-start prediction latency.

- **Expand Knowledge Graph Coverage Across the Full Medical Curriculum:** Scale the domain knowledge extraction and graph construction pipeline beyond the initial foundational textbooks (physiology, biochemistry, and histology) to encompass the complete preclinical and clinical medical curriculum—incorporating pharmacology, pathology, microbiology, internal medicine, and surgery into unified topic hierarchies and prerequisite dependency networks.

- **Establish End-to-End Clinical and Quantitative Benchmark Validation:** Conduct comprehensive expert evaluations with medical educators to audit domain knowledge graphs, quantitatively benchmark GraphRAG retrieval performance (e.g., Hit@K, MRR, and context groundedness), and perform psychometric item analyses on generated MCQs to ensure clinical accuracy.

- **Implement Multilingual Support and Indonesian Medical NLP:** Extend the ingestion, knowledge extraction, and MCQ generation pipelines to support Bahasa Indonesia by integrating localized medical terminologies and ontology mappings (such as SNOMED CT Indonesian translations), fine-tuning bilingual extraction prompts, and providing full localization across the iPadOS application interface.

- **Integrate Multimodal and Image-Grounded Question Synthesis:** Connect the existing multimodal preprocessing pipeline (document figure parsing, vision-language captioning, and numbered label masking) directly into the downstream MCQ generation engine and GraphRAG retrieval framework. This will enable automated synthesis of visual diagnostic questions, diagram-label identification tasks, and hot-spot anatomical flashcards.

- **Expand Question Typologies and Calibrate Cognitive Difficulty:** Introduce a wider variety of question structures, including multi-step clinical reasoning vignettes, case-based scenarios, and assertion-reason questions, while retaining the five-option multiple-choice format. Incorporate cognitive taxonomy classifiers (such as Bloom's Revised Taxonomy) to enable more precise difficulty calibration and stronger alignment with medical board examination standards.

- **Upgrade Student Mastery Modeling with Deep Knowledge Tracing:** Enhance the mastery inference framework by transitioning from standard single-parameter BKT toward multi-signal or Deep Knowledge Tracing (DKT) architectures, incorporating continuous behavioral signals—such as response latency, self-assessed confidence scores, circadian study intervals, and knowledge graph prerequisite traversal—to yield significantly more accurate student models.

- **Broaden Ingestion Pipelines Across Multi-Format Academic Documents:** Extend the preprocessing and OCR ingestion layer beyond textbook PDFs to handle additional institutional formats, including Microsoft Word (.docx) lecture notes, slide presentations (.pptx), clinical practice guidelines, web articles, and scanned paper-based exam archives with formula and tabular data recovery.